<<OwlyFly V1.0>>

mainmenu.gd:

```
extends Control
const UPDATE_JSON_URL: String =
"https://inkalatia.github.io/updateowlyfly.json/update.json"
const CURRENT_VERSION: int = 2
var languages = {
	0: {  # English
		"Play": " Quick Play ",
		"Extra1": " Extra levels ",
		"play_colors": "  Colors ",
		"play_letters": "  Letters ",
		"play_shapes": "  Shapes ",
		"play_custom": "Play Custom",
		"options": "Options",
		"shop": "Shop",
		"aimier": "Aimier KG",
		"demo": " Demo(Expires on restart) ",
		"exit": "Exit",
		"update_message": "A new version is available! Please update for the best
experience.",
		"update_button": "Update Now",
		"later_button": "Later",
	},
	1: {  # Chinese
		"Play": " 快速游戏 ",
		"Extra1": " 额外关卡 ",
		"play_colors": " 玩颜色 ",
		"play_letters": " 玩字母 ",
		"play_shapes": " 玩形状 ",
		"play_custom": "玩自定义",
		"options": "选项",
		"shop": "游戏商店",
		"aimier": " 爱弥儿幼儿园 ",
		"demo": " 试玩版 (重启失效) ",
		"exit": "退出",
		"update_message": "有新版本可用！请更新以获得最佳体验。",
		"update_button": "立即更新",
		"later_button": "稍后",
	}
}
var current_language
var pressed
var english_theme = preload("res://resources/EnglishButtonSize.tres")
var chinese_theme = preload("res://resources/ChineseButtonSize.tres")
var http_request: HTTPRequest
var update_available: bool = false
var update_url: String = ""
func _ready()-> void:
	if Global.aimierkg==true:
		$VBoxContainer/AimierKGButton.visible=true
	if Global.demo==true:
		$VBoxContainer/DemoButton.visible=true
	pressed=0
	current_language = Global.language
```

<<OwlyFly V1.0>>

```
	update_language()  # Set the initial language
	Global.inmenu=true
	$AudioSFX.volume_db = linear_to_db(Global.voice_volume / 10.0)
	#scale_fonts_recursive(self, 0.5)
	http_request = HTTPRequest.new()
	add_child(http_request)
	http_request.request_completed.connect(_on_http_request_completed)
	# Start the update check
	check_for_updates()
func check_for_updates() -> void:
	# Request the update JSON file
	var error = http_request.request(UPDATE_JSON_URL)
	if error != OK:
		print("Error requesting update information")
func _on_http_request_completed(result: int, response_code: int, headers:
PackedStringArray, body: PackedByteArray) -> void:
	if result != HTTPRequest.RESULT_SUCCESS:
		push_error("Update check failed: " + str(result))
		return
	if response_code != 200:
		print("Update check failed with status: " + str(response_code))
		return
	var json = JSON.new()
	var parse_result = json.parse(body.get_string_from_utf8())
	if parse_result != OK:
		print("JSON parse error: " + json.get_error_message())
		return
	var data = json.get_data()
	if data and data.has("latest_version") and data["latest_version"] >
CURRENT_VERSION:
		update_available = true
		update_url = data.get("url", "https://your-itch-page.itch.io/your-game")
		show_update_notification()
func show_update_notification() -> void:
	if not update_available:
		return
	# Get the current theme based on language
	var current_theme = english_theme if current_language == 0 else chinese_theme
	# Create update notification UI
	var popup = Window.new()
	popup.title = "Update Available"
	popup.size = Vector2(800, 400)  # Increased size for larger text
	popup.unresizable = true
	popup.exclusive = true
	var container = VBoxContainer.new()
	container.size_flags_vertical = Control.SIZE_EXPAND_FILL
	container.alignment = BoxContainer.ALIGNMENT_CENTER
	container.add_theme_constant_override("separation", 30)  # Add more spacing
	popup.add_child(container)
	var message = Label.new()
	message.theme = current_theme  # Apply the current theme
	message.text = languages[current_language].get(
		"update_message",
		"A new version is available! Please update for the best experience."
	)
	message.autowrap_mode = TextServer.AUTOWRAP_WORD
```

<<OwlyFly V1.0>>

```
	message.horizontal_alignment = HORIZONTAL_ALIGNMENT_CENTER
	message.size_flags_horizontal = Control.SIZE_EXPAND_FILL
	message.vertical_alignment = VERTICAL_ALIGNMENT_CENTER
	message.custom_minimum_size = Vector2(700, 150)  # Larger text area
	# Double the font size for the label
	var base_font_size = message.get_theme_font_size("font_size")
	message.add_theme_font_size_override("font_size", base_font_size * 2)
	container.add_child(message)
	var button_container = HBoxContainer.new()
	button_container.alignment = BoxContainer.ALIGNMENT_CENTER
	button_container.add_theme_constant_override("separation", 50)  # More space
between buttons
	container.add_child(button_container)
	var update_button = Button.new()
	update_button.theme = current_theme  # Apply the current theme
	update_button.text = languages[current_language].get("update_button", "Update Now")
	update_button.custom_minimum_size = Vector2(250, 80)  # Larger button size
	# Double the font size for the button
	var button_font_size = update_button.get_theme_font_size("font_size")
	update_button.add_theme_font_size_override("font_size", button_font_size * 2)
	update_button.pressed.connect(_on_update_pressed)
	button_container.add_child(update_button)
	var later_button = Button.new()
	later_button.theme = current_theme  # Apply the current theme
	later_button.text = languages[current_language].get("later_button", "Later")
	later_button.custom_minimum_size = Vector2(250, 80)  # Larger button size
	# Double the font size for the button
	later_button.add_theme_font_size_override("font_size", button_font_size * 2)
	later_button.pressed.connect(popup.queue_free)
	button_container.add_child(later_button)
	add_child(popup)
	popup.popup_centered()
func _on_update_pressed() -> void:
	# Open the itch.io page in the browser
	OS.shell_open(update_url)
	# Close the game to encourage updating
	get_tree().quit()
func scale_fonts_recursive(node, scale_factor):
	if node is Control:
		# Adjust font size for labels and buttons
		if node is Label or node is Button:
			var current_font_size = node.get("theme_override_font_sizes/font_size")
			if current_font_size != null:
	node.set("theme_override_font_sizes/font_size", int(current_font_size *
scale_factor))
	# Recursively process children
	for child in node.get_children():
		scale_fonts_recursive(child, scale_factor)
func update_language():
	print("current global language is: "+ str(Global.language) + "current language: "+
str(current_language))
	$VBoxContainer/PlayButton.text = languages[current_language]["Play"]
	$VBoxContainer/PlayExtraButton.text = languages[current_language]["Extra1"]
	$VBoxContainer/PlayShapesButton.text = languages[current_language]["play_shapes"]
	$VBoxContainer/PlayCustomButton.text = languages[current_language]["play_custom"]
	$VBoxContainer/OptionsButton.text = languages[current_language]["options"]
```

```
	$VBoxContainer/ShopButton.text = languages[current_language]["shop"]
	$VBoxContainer/ExitButton.text = languages[current_language]["exit"]
	$VBoxContainer/BasicPlay/Colors.text = languages[current_language]["play_colors"]
	$VBoxContainer/BasicPlay/Letters.text = languages[current_language]["play_letters"]
	$VBoxContainer/BasicPlay/Shapes.text = languages[current_language]["play_shapes"]
	$VBoxContainer/AimierKGButton.text = languages[current_language]["aimier"]
	$VBoxContainer/DemoButton.text = languages[current_language]["demo"]
	if current_language ==0:
		$LanguageButton.icon=preload("res://images/chinese.png")
		$VBoxContainer/PlayButton.theme=english_theme
		$VBoxContainer/PlayExtraButton.theme=english_theme
		$VBoxContainer/PlayShapesButton.theme=english_theme
		$VBoxContainer/AimierKGButton.theme=english_theme
		$VBoxContainer/PlayCustomButton.theme=english_theme
		$VBoxContainer/OptionsButton.theme=english_theme
		$VBoxContainer/ShopButton.theme=english_theme
		$VBoxContainer/ExitButton.theme=english_theme
		$VBoxContainer/BasicPlay/Colors.theme=english_theme
		$VBoxContainer/BasicPlay/Letters.theme=english_theme
		$VBoxContainer/BasicPlay/Shapes.theme=english_theme
		$VBoxContainer/AimierKGButton.theme=english_theme
		$VBoxContainer/DemoButton.theme=english_theme
	else:
		$LanguageButton.icon=preload("res://images/english.png")
		$VBoxContainer/PlayButton.theme=chinese_theme
		$VBoxContainer/PlayExtraButton.theme=chinese_theme
		$VBoxContainer/PlayShapesButton.theme=chinese_theme
		$VBoxContainer/AimierKGButton.theme=chinese_theme
		$VBoxContainer/PlayCustomButton.theme=chinese_theme
		$VBoxContainer/OptionsButton.theme=chinese_theme
		$VBoxContainer/ShopButton.theme=chinese_theme
		$VBoxContainer/ExitButton.theme=chinese_theme
		$VBoxContainer/BasicPlay/Colors.theme=chinese_theme
		$VBoxContainer/BasicPlay/Letters.theme=chinese_theme
		$VBoxContainer/BasicPlay/Shapes.theme=chinese_theme
		$VBoxContainer/AimierKGButton.theme=chinese_theme
		$VBoxContainer/DemoButton.theme=chinese_theme
func _on_language_button_pressed() -> void:
	if current_language ==0:
		current_language=1
		$LanguageButton.icon=preload("res://images/chinese.png")
	else:
		current_language=0
		$LanguageButton.icon=preload("res://images/english.png")
	Global.language=current_language
	update_language()
	Global.save_settings()
# Update the _create_settings_resource to preserve purchases
func _create_settings_resource() -> Resource:
	var settings = SettingsResource.new()
	if ResourceLoader.exists("user://settings.tres"):
		settings = ResourceLoader.load("user://settings.tres")
	# Only update these specific settings
	settings.difficulty = Global.difficulty
	settings.voice_volume = Global.voice_volume
	settings.music_volume = Global.music_volume
```

<<OwlyFly V1.0>>

```
	settings.target_count = Global.target_count
	settings.is_premium_user = Global.is_premium_user
	settings.screen_limits = Global.screen_limits
	settings.movement = Global.movement
	settings.language = Global.language
	settings.aimier = Global.aimierkg
	settings.purchased_landscapes = Global.purchased_landscapes
	settings.purchased_skins = Global.purchased_skins
	return settings
func alloptionshidden():
	$VBoxContainer/BasicPlay.visible=false
func _on_exit_button_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
		get_tree().quit()
func _on_play_button_pressed() -> void:
	alloptionshidden()
	$VBoxContainer/BasicPlay.visible=true
	$AudioSFX.play()
	await $AudioSFX.finished
func _on_colors_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
		Global.game_mode = "colors" # Set the game mode to colors
		Global.unit=["main"]
		get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_letters_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
		Global.game_mode = "letters" # Set the game mode to letters
		Global.unit=["main"]
	get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_shapes_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
		Global.game_mode = "shapes" # Set the game mode to letters
		Global.unit=["main"]
		get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_play_extra_button_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
	get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
func _on_play_shapes_button_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
```

```
            await $AudioSFX.finished
            Global.game_mode = "shapes" # Set the game mode to letters
      get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_play_custom_button_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/customize_game.tscn")
func _on_options_button_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/options.tscn")
func _on_aimier_kg_button_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/aimier_kg.tscn")
func _on_shop_button_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
            get_tree().change_scene_to_file("res://scenes/shop.tscn")
func _on_demo_button_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
            get_tree().change_scene_to_file("res://scenes/demo.tscn")
```

global.gd:

```
extends Node
# Define global variables and configurations
var target_color: Color = Color(0, 1, 0)  # Initial target color (green)
var target_letter: String = "A"  # Initial target letter
var target_shape: String = "Circle"  # Initial target shape
var good_obstacles_collected = 0  # Counter for good obstacles collected
var target_count = 10  # Number of obstacles to collect to win
var segment_counter = 0
var game_speed = 40.0  # Define the global speed variable
var difficulty = 1  # Difficulty level
var game_mode = "colors"  # Track the current game mode
var current_topic: String = ""  # Track the current custom topic
var is_premium_user: bool = false  # Track if the user is a premium user
var topics_created: int = 0  # Track number of topics created
var voice_volume = 10
var music_volume = 10
var inmenu = true
var screen_limits = false
var victory = false
var scene = 0
var showad=false
var movement = 1
```

```
var language = 1
var aimierkg = false
var demo = false
var region = "China"
var equipped_skin: int = 0  # Stores selected skin index
var selected_landscape: int = 0
var current_column: int = 1
var current_row: int = 0
var unit: Array
var purchased_skins:= {0: true}
var purchased_landscapes:= {}
var savememory=false
var difficulty1v1=1
var versusdata: Array = []
var topic = {
      "Flower": {"name": "Flower", "image":
preload("res://images/babyclass/unit2/flower.png"), "audio":
preload("res://images/babyclass/unit2/flower.mp3")},
     "Lake": {"name": "Lake", "image": preload("res://images/babyclass/unit2/lake.jpeg"),
"audio": preload("res://images/babyclass/unit2/lake.mp3")},
      "Moon": {"name": "Moon", "image": preload("res://images/babyclass/unit2/moon.png"),
"audio": preload("res://images/babyclass/unit2/moon.mp3")},
      "Plant": {"name": "Plant", "image":
preload("res://images/babyclass/unit2/plant.png"), "audio":
preload("res://images/babyclass/unit2/plant.mp3")},
      "Rainbow": {"name": "Rainbow", "image":
preload("res://images/babyclass/unit2/rainbow.jpeg"), "audio":
preload("res://images/babyclass/unit2/rainbow.mp3")},
      "Star": {"name": "Star", "image": preload("res://images/babyclass/unit2/star.png"),
"audio": preload("res://images/babyclass/unit2/star.mp3")},
      "Sun": {"name": "Sun", "image": preload("res://images/babyclass/unit2/sun.png"),
"audio": preload("res://images/babyclass/unit2/sun.mp3")},
      "Tree": {"name": "Tree", "image": preload("res://images/babyclass/unit2/tree.png"),
"audio": preload("res://images/babyclass/unit2/tree.mp3")},
}
# List of color names and corresponding audio files
var color_data = {
      Color(0, 1, 0): {"name": "Green", "audio": preload("res://audio/green.mp3")},
      Color(0, 0, 1): {"name": "Blue", "audio": preload("res://audio/blue.mp3")},
      Color(1, 0, 0): {"name": "Red", "audio": preload("res://audio/red.mp3")},
      Color(1, 1, 0): {"name": "Yellow", "audio": preload("res://audio/yellow.mp3")},
      Color(1, 0.5, 0): {"name": "Orange", "audio": preload("res://audio/orange.mp3")},
      Color(0.5, 0, 1): {"name": "Purple", "audio": preload("res://audio/purple.mp3")},
      Color(1, 0.08, 0.58): {"name": "Pink", "audio": preload("res://audio/pink.mp3")},
      Color(0.5, 0.25, 0): {"name": "Brown", "audio": preload("res://audio/brown.mp3")},
      Color(0, 0, 0): {"name": "Black", "audio": preload("res://audio/black.mp3")},
      Color(1, 1, 1): {"name": "White", "audio": preload("res://audio/white.mp3")},
      Color(0.5, 0.5, 0.5): {"name": "Gray", "audio": preload("res://audio/gray.mp3")}
}
# List of letters and corresponding audio files
var letter_data = {
      "A": {"name": "A", "audio": preload("res://audio/a.mp3")},
      "B": {"name": "B", "audio": preload("res://audio/b.mp3")},
      "C": {"name": "C", "audio": preload("res://audio/c.mp3")},
      "D": {"name": "D", "audio": preload("res://audio/d.mp3")},
      "E": {"name": "E", "audio": preload("res://audio/e.mp3")},
```

```
	"F": {"name": "F", "audio": preload("res://audio/f.mp3")},
	"G": {"name": "G", "audio": preload("res://audio/g.mp3")},
	"H": {"name": "H", "audio": preload("res://audio/h.mp3")},
	"I": {"name": "I", "audio": preload("res://audio/i.mp3")},
	"J": {"name": "J", "audio": preload("res://audio/j.mp3")},
	"K": {"name": "K", "audio": preload("res://audio/k.mp3")},
	"L": {"name": "L", "audio": preload("res://audio/l.mp3")},
	"M": {"name": "M", "audio": preload("res://audio/m.mp3")},
	"N": {"name": "N", "audio": preload("res://audio/n.mp3")},
	"O": {"name": "O", "audio": preload("res://audio/o.mp3")},
	"P": {"name": "P", "audio": preload("res://audio/p.mp3")},
	"Q": {"name": "Q", "audio": preload("res://audio/q.mp3")},
	"R": {"name": "R", "audio": preload("res://audio/r.mp3")},
	"S": {"name": "S", "audio": preload("res://audio/s.mp3")},
	"T": {"name": "T", "audio": preload("res://audio/t.mp3")},
	"U": {"name": "U", "audio": preload("res://audio/u.mp3")},
	"V": {"name": "V", "audio": preload("res://audio/v.mp3")},
	"W": {"name": "W", "audio": preload("res://audio/w.mp3")},
	"X": {"name": "X", "audio": preload("res://audio/x.mp3")},
	"Y": {"name": "Y", "audio": preload("res://audio/y.mp3")},
	"Z": {"name": "Z", "audio": preload("res://audio/z.mp3")},
}
# List of shapes, corresponding images, and audio files
var shape_data = {
	"Circle": {"name": "Circle", "image": preload("res://images/circle.png"), "audio":
preload("res://audio/circle.mp3")},
	"Square": {"name": "Square", "image": preload("res://images/square.png"), "audio":
preload("res://audio/square.mp3")},
	"Triangle": {"name": "Triangle", "image": preload("res://images/triangle.jpeg"),
"audio": preload("res://audio/triangle.mp3")},
	"Oval": {"name": "Oval", "image": preload("res://images/oval.jpeg"), "audio":
preload("res://audio/oval.mp3")},
	"Rectangle": {"name": "Rectangle", "image": preload("res://images/rectangle.jpeg"),
"audio": preload("res://audio/rectangle.mp3")},
	"Diamond": {"name": "Diamond", "image": preload("res://images/diamond.jpeg"),
"audio": preload("res://audio/diamond.mp3")},
	"Heart": {"name": "Heart", "image": preload("res://images/heart.jpeg"), "audio":
preload("res://audio/heart.mp3")},
	"Pentagon": {"name": "Pentagon", "image": preload("res://images/pentagon.jpg"),
"audio": preload("res://audio/pentagon.mp3")},
	"Star": {"name": "Star", "image": preload("res://images/star.jpeg"), "audio":
preload("res://audio/star.mp3")},
	# Add more shapes and audio files as needed
}
# Dictionary to store custom topics
var custom_topics = {}
var democlass = false
func _ready() -> void:
	#var user_dir = OS.get_user_data_dir()
	#var settings_path = user_dir.path_join("settings.tres")
	#
	## Print the path to the console
	#print("Settings file path: ", settings_path)
	var settings = ResourceLoader.load("user://settings.tres")
	if settings:
		settings = settings as SettingsResource
```

<<OwlyFly V1.0>>

```
            difficulty = settings.difficulty
            voice_volume = settings.voice_volume
            music_volume = settings.music_volume
            target_count = settings.target_count
            is_premium_user = settings.is_premium_user
            screen_limits = settings.screen_limits
            movement = settings.movement
            language = settings.language
            aimierkg = settings.aimier
            equipped_skin = settings.equipped_skin
            purchased_skins = settings.purchased_skins
            purchased_landscapes = settings.purchased_landscapes
      else:
            save_settings()
func save_settings():
      # Load existing settings first
      var settings
      if ResourceLoader.exists("user://settings.tres"):
            settings = ResourceLoader.load("user://settings.tres")
      else:
            settings = SettingsResource.new()
      # Update settings with current values
      settings.difficulty = difficulty
      settings.voice_volume = voice_volume
      settings.music_volume = music_volume
      settings.target_count = target_count
      settings.is_premium_user = is_premium_user
      settings.screen_limits = screen_limits
      settings.movement = movement
      settings.language = language
      settings.aimier = aimierkg
      settings.purchased_skins = purchased_skins
      settings.purchased_landscapes = purchased_landscapes
      # Save the updated settings
      var error = ResourceSaver.save(settings, "user://settings.tres")
```

choose_landscape.gd:

```
extends Control
@onready var model_holder: Node3D =
$CanvasLayer/VBoxContainer/HBoxContainer/SubViewportContainer/SubViewport/ModelHolder
@onready var arrow_left_skin: Button =
$CanvasLayer/VBoxContainer/HBoxContainer/ArrowLeftSkin
@onready var arrow_right_skin: Button =
$CanvasLayer/VBoxContainer/HBoxContainer/ArrowRightSkin
@export var offset: float = 30.0
@export var scale_between_cards: float = 0.05
@export var anim_duration: float = 0.45
@export var anim_duration_offset: float = 0.15
@onready var cards_container: Container = $CanvasLayer/Cards
@onready var top_point: Marker2D = $CanvasLayer/TopPoint
@onready var popup_content: Control = $SettingsPopup/PopupContent
# Add these references to the popup elements
@onready var settings_panel: PanelContainer = $SettingsPanel
@onready var lowmode_button: CheckButton = $MarginContainer/VBoxContainer/CheckButton
@onready var difficulty_button: Button = $SettingsPanel/VBoxContainer/Difficulty
@onready var movement_mode_button: Button = $SettingsPanel/VBoxContainer/MovementMode
@onready var screen_limits_button: Button = $SettingsPanel/VBoxContainer/ScreenLimits
```

<<OwlyFly V1.0>>

```
@onready var win_condition_label: Label = $SettingsPanel/VBoxContainer/WinConditionLabel
@onready var settings_button: Button = $MarginContainer/VBoxContainer/SettingsButton
@onready var close_button: Button = $SettingsPanel/VBoxContainer/CloseButton
var flashing_areas_scene = preload("res://scenes/flashing_areas.tscn")
var flashing_areas_scene_2 = preload("res://scenes/flashing_areas2.tscn")
var rotate_speed = 2.0
var is_dragging = false
var last_mouse_pos: Vector2
var swipe_start: Vector2
var lowmode
var flashing_areas_instance
var languages = {
      0: {  # English
            "Play": "   Play   ",
            "Skins": "SKINS",
            "Choose": "LANDSCAPE", #Choose a level",
            "NoPremium": "Available for premium mode only",
            "MainMenu": "  Main Menu  ",
            "NoSkins": "No available skins",
            "LowMode": "Low Mode",
            "Difficulty": "Difficulty (",
            "MovementMode": "Movement: TOUCH",
            "MovementMode2": "Movement: SWIPE",
            "ScreenLimits": " Help: OFF ",
            "ScreenLimits2": " Help: ON ",
            "WinconditionLabel": " Items to Collect: ",
            "Settings": "Settings"  # ADDED
      },
      1: {  # Chinese
            "Play": " 开始游戏 ",
            "Skins": "皮肤",
            "Choose": "场景", #"选择一个关卡",
            "NoPremium": "仅限高级模式使用",
            "MainMenu": "   主菜单   ",
            "NoSkins": "没有可用皮肤",
            "LowMode": "低画质模式",
            "Difficulty": "难度 (",
            "MovementMode": "移动模式：触摸",
            "MovementMode2": "移动模式：滑动",
            "ScreenLimits": "屏幕帮助：关闭",
            "ScreenLimits2": "屏幕帮助：开启",
            "WinconditionLabel": "需要收集的物品数量：",
            "Settings": "设置"  # ADDED
      }
}
var current_language = Global.language
var pressed
var available_skins: Array = []
var current_skin_index := 0
var settings: SettingsResource
var first_card_pos: Vector2
var first_card_scale: Vector2
var tween: Tween
var tween_last_card: Tween
```

<<OwlyFly V1.0>>

```
var landscapes: Array = []
var selected_landscape_index: int = 0
func _ready() -> void:
	settings_panel.visible = false
	lowmode  = $CanvasLayer/TextureRect
	Global.savememory=false
	lowmode.visible=false
	load_settings()
	initialize_skins()
	update_skin_display()
	initialize_landscapes()
	create_landscape_cards()
	setup_initial_positions()
	update_cards()
	pressed=0
	# Setup settings button
	settings_button.text = languages[current_language]["Settings"]
	settings_button.pressed.connect(_on_settings_button_pressed)
	# Setup popup controls
	lowmode_button.text = languages[current_language]["LowMode"]
	difficulty_button.text = languages[current_language]["Difficulty"] +
str(Global.difficulty) + ")"
	movement_mode_button.text = languages[current_language]["MovementMode"] if
Global.movement == 0 else languages[current_language]["MovementMode2"]
	screen_limits_button.text = languages[current_language]["ScreenLimits"]
if !Global.screen_limits else languages[current_language]["ScreenLimits2"]
	win_condition_label.text = languages[current_language]["WinconditionLabel"] +
str(Global.target_count)
	# Connect popup signals
	lowmode_button.toggled.connect(_on_check_button_toggled)
	difficulty_button.pressed.connect(_on_difficulty_pressed)
	movement_mode_button.pressed.connect(_on_movement_mode_pressed)
	screen_limits_button.pressed.connect(_on_screen_limits_pressed)
	# Other UI setup
	$CanvasLayer/VBoxContainer/Landscape/Label.text =
languages[current_language]["Choose"]
	$MarginContainer/VBoxContainer/MainMenu.text =
languages[current_language]["MainMenu"]
	$MarginContainer/VBoxContainer/PlayButton.text =
languages[current_language]["Play"]
	$CanvasLayer/VBoxContainer/Skins/Label.text = languages[current_language]["Skins"]
	$AudioSFX.volume_db = linear_to_db(Global.voice_volume / 10.0)
	if landscapes.size() > 0:
		Global.selected_landscape = landscapes[0]["scene"]
	await get_tree().process_frame
	_update_selected_landscape_from_top_card()
	setup_initial_positions()
	update_cards()
	cards_container.queue_redraw()
#Settings button handler
func _on_settings_button_pressed() -> void:
	$AudioSFX.play()
	if settings_panel.visible:
		# Hide animation
		var tween_hide = create_tween()
		tween_hide.tween_property(settings_panel, "modulate", Color(1, 1, 1, 0), 0.2)
```

```
            tween_hide.tween_property(settings_panel, "scale", Vector2(0.9, 0.9), 0.2)
            tween_hide.tween_callback(func(): settings_panel.visible = false)
      else:
            # Show animation with bounce
            settings_panel.visible = true
            settings_panel.modulate = Color(1, 1, 1, 0)
            settings_panel.scale = Vector2(0.8, 0.8)
            var tween_show = create_tween().set_parallel(true)
            tween_show.tween_property(settings_panel, "modulate", Color(1, 1, 1, 1),
0.3).set_ease(Tween.EASE_OUT)
            # Bounce effect
            tween_show.tween_property(settings_panel, "scale", Vector2(1.1, 1.1),
0.15).set_ease(Tween.EASE_OUT)
            tween_show.tween_property(settings_panel, "scale", Vector2(0.95, 0.95),
0.1).set_delay(0.15).set_ease(Tween.EASE_IN)
            tween_show.tween_property(settings_panel, "scale", Vector2(1.0, 1.0),
0.1).set_delay(0.25).set_ease(Tween.EASE_OUT)
func create_landscape_cards():
      if not cards_container:
            push_error("Cards container not found!")
            return
      # Clear existing cards first
      for child in cards_container.get_children():
            child.queue_free()
      # Create temporary array with desired initial order
      var ordered_landscapes = []
      # 1. Add Magic Forest first (will be bottom card)
      ordered_landscapes.append({
            "name": "MagicForest",
            "preview": preload("res://images/fantasy forest.png"),
            "scene": 0
      })
      # 2. Add other default landscapes
      ordered_landscapes.append({
            "name": "City1",
            "preview": preload("res://images/city.png"),
            "scene": 1
      })
      ordered_landscapes.append({
            "name": "Pirate",
            "preview": preload("res://images/pirate.png"),
            "scene": 2
      })
      # 3. Add purchased landscapes
      for shop_index in settings.purchased_landscapes:
            if settings.purchased_landscapes[shop_index]:
                  var landscape = get_landscape_data(shop_index + 3)
                  if landscape:
                        ordered_landscapes.append(landscape)
      # Add cards in reverse order to get Magic Forest on top
      for i in range(ordered_landscapes.size()-1, -1, -1):
            var landscape = ordered_landscapes[i]
            if landscape["scene"] >= 3 and not
settings.purchased_landscapes.get(landscape["scene"] - 3, false):
                  continue
            var new_card = preload("res://scenes/cards_stack/card.tscn").instantiate()
```

<<OwlyFly V1.0>>

```
            new_card.size_flags_horizontal = Control.SIZE_EXPAND_FILL
            new_card.size_flags_vertical = Control.SIZE_EXPAND_FILL
            if new_card.has_node("Preview"):
                  new_card.get_node("Preview").texture = landscape["preview"]
      new_card.put_back.connect(on_landscape_selected.bind(landscape["scene"]))
            cards_container.add_child(new_card)
      # Force initial selection update
      _update_selected_landscape_from_top_card()
func _update_selected_landscape_from_top_card():
      if cards_container.get_child_count() == 0:
            return
      var top_card = cards_container.get_child(cards_container.get_child_count() - 1)
      for landscape in landscapes:
            if top_card.get_node("Preview").texture == landscape["preview"]:
                  Global.selected_landscape = landscape["scene"]
                  #$VBoxContainer/Landscape/Label.text = "%s: %s" % [
                        #languages[current_language]["Choose"],
                        #landscape["name"]
                  #]
                  break
func _input(event):
      # Get reference to the viewport container
      var viewport_container =
$CanvasLayer/VBoxContainer/HBoxContainer/SubViewportContainer
      var viewport_rect = viewport_container.get_global_rect()
      # Handle both mouse and touch input the same way
      if event is InputEventMouseButton or event is InputEventScreenTouch:
            if viewport_rect.has_point(event.position):
                  is_dragging = event.pressed
                  last_mouse_pos = event.position
      elif event is InputEventMouseMotion or event is InputEventScreenDrag:
            if is_dragging and viewport_rect.has_point(event.position):
                  var delta = (event.position - last_mouse_pos).x
                  model_holder.rotate_y(delta * 0.01)
                  last_mouse_pos = event.position
func _on_main_menu_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_play_button_pressed() -> void:
      if pressed==0:
            pressed=1
            Global.scene=  Global.selected_landscape
            Global.equipped_skin = available_skins[current_skin_index]
            $AudioSFX.play()
            await $AudioSFX.finished
            get_tree().change_scene_to_file("res://KidsGame.tscn")
func load_settings():
      if ResourceLoader.exists("user://settings.tres"):
            settings = ResourceLoader.load("user://settings.tres")
      else:
            settings = SettingsResource.new()
            settings.purchased_skins = {0: true}  # Default skin
func initialize_skins():
```

<<OwlyFly V1.0>>

```
	# Create list of available skins (default + purchased)
	available_skins = []
	# Always include default skin
	available_skins.append(0)
	# Add other purchased skins
	for index in settings.purchased_skins:
		if index != 0 && settings.purchased_skins[index]:
			available_skins.append(index)
	available_skins.sort()
	update_arrow_visibility()
func update_arrow_visibility():
	arrow_left_skin.visible = available_skins.size() > 1
	arrow_right_skin.visible = available_skins.size() > 1
func update_skin_display():
	clear_model_holder()
	if available_skins.is_empty():
		$VBoxContainer/Skins/Label.text = languages[current_language]["NoSkins"]
		return
	var skin_index = available_skins[current_skin_index]
	var skin_scene = get_skin_scene(skin_index)
	var skin_instance = skin_scene.instantiate()
	model_holder.add_child(skin_instance)
	print(skin_index)
	# Play animation if exists
	var anim_player = skin_instance.find_child("AnimationPlayer")
	if anim_player and anim_player.get_animation_list().size() > 0:
		anim_player.play(anim_player.get_animation_list()[0])
		anim_player.speed_scale = 1.0
		if skin_index==3: #this is the index of the red dragon
				anim_player.speed_scale = 2.0
func get_skin_scene(index: int) -> PackedScene:
	# skin scene loading logic
	match index:
		0: return preload("res://3DModels/owl/owl.glb")
		1: return preload("res://3DModels/ww1plane/ww1_plane.glb")
		2: return preload("res://3DModels/eagle/white_eagle.glb")
		3: return preload("res://3DModels/dragon1/dragon_flying.glb")
		4: return preload("res://3DModels/bat/emerald_bat.glb")
		5: return preload("res://3DModels/dragon2/european_dragon.glb")
		# Add other skins...
		_: return preload("res://3DModels/owl/owl.glb")  # Fallback
func clear_model_holder():
	for child in model_holder.get_children():
		child.queue_free()
func _on_arrow_left_skin_pressed() -> void:
	current_skin_index = wrapi(current_skin_index - 1, 0, available_skins.size())
	Global.equipped_skin = available_skins[current_skin_index]
	update_skin_display()
func _on_arrow_right_skin_pressed() -> void:
	current_skin_index = wrapi(current_skin_index + 1, 0, available_skins.size())
	Global.equipped_skin = available_skins[current_skin_index]
	update_skin_display()
func initialize_landscapes():
	# Always include default landscapes (indexes 0-2)
	landscapes = [
```

<<OwlyFly V1.0>>

```
            {"name": "Magic Forest", "preview": preload("res://images/fantasy
forest.png"), "scene": 0},
            {"name": "City", "preview": preload("res://images/city.png"), "scene": 1},
            {"name": "Forest", "preview": preload("res://images/pirate.png"), "scene": 2}
      ]
      # Add purchased premium landscapes (indexes 3-5)
      for shop_index in settings.purchased_landscapes:
            if settings.purchased_landscapes[shop_index]:
                  # Map shop index (0-2) to actual landscape index (3-5)
                  var actual_index = shop_index + 3
                  var landscape = get_landscape_data(actual_index)
                  if landscape and not landscape in landscapes:
                        landscapes.append(landscape)
func get_landscape_data(index: int) -> Dictionary:
      match index:
            3: return {"name": "Cathedral", "preview":
preload("res://images/cathedral.jpg"), "scene": 3}
            4: return {"name": "City Tunnel", "preview":
preload("res://images/citytunnel2.png"), "scene": 4}
            5: return {"name": "Blossom Town", "preview":
preload("res://images/blossomtown2.png"), "scene": 5}
            _: return {}
func setup_initial_positions():
      if cards_container.get_child_count() == 0:
            return
      # Center of the container
      first_card_pos = Vector2(
            cards_container.size.x / 2 - cards_container.get_child(0).size.x / 2,
            cards_container.size.y / 2
      )
      first_card_scale = Vector2(1, 1)
func update_cards() -> void:
      if not cards_container or cards_container.get_child_count() == 0:
            return
      if tween and tween.is_running():
            tween.kill()
      tween =
create_tween().set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_CUBIC).set_parallel(true
)
      var child_count = cards_container.get_child_count()
      var base_position = cards_container.size / 2  # Center of container
      for i in child_count:
            var card = cards_container.get_child(i)
            var depth = child_count - i - 1  # Reverse order for stacking
            # Vertical offset with decreasing scale
            var target_y = base_position.y - (offset * depth)
            var target_scale = Vector2(1.0, 1.0) - (Vector2.ONE * scale_between_cards *
depth)
            tween.tween_property(card, "position:y", target_y, anim_duration)
            tween.tween_property(card, "scale", target_scale, anim_duration)
            # Keep X centered
            tween.parallel().tween_property(card, "position:x", base_position.x -
card.size.x/2, anim_duration)
func on_landscape_selected(scene_index: int) -> void:
      Global.selected_landscape = scene_index
      # Visual feedback for selection
```

<<OwlyFly V1.0>>

```
      #$VBoxContainer/Landscape/Label.text = "%s: %d" %
[languages[current_language]["Choose"], scene_index]
      animate_card_selection()
func animate_card_selection():
      if cards_container.get_child_count() == 0:
            return
      var selected_card = cards_container.get_child(cards_container.get_child_count() -
1)
      if tween_last_card and tween_last_card.is_running():
            tween_last_card.kill()
      # Calculate target position relative to TopPoint
      var target_pos = top_point.global_position - selected_card.global_position
      target_pos = selected_card.global_position + target_pos
      tween_last_card = create_tween()
      tween_last_card.tween_property(selected_card, "global_position", target_pos,
anim_duration/2)
     tween_last_card.parallel().tween_property(selected_card, "scale", Vector2(0.8, 0.8),
anim_duration/2)
      tween_last_card.tween_callback(cycle_cards)
func cycle_cards():
      if cards_container.get_child_count() <= 1:
            return
      # Move top card to bottom
      var last_card = cards_container.get_child(cards_container.get_child_count() - 1)
      cards_container.move_child(last_card, 0)
      # Reset all card properties
      setup_initial_positions()
      update_cards()
      # Update selection after animation
      await tween.finished
      _update_selected_landscape_from_top_card()
func set_cards_3D(state: bool) -> void:
      for c in cards_container.get_children():
            c.is_3D = true  # Force 3D mode always
func _on_check_button_toggled(toggled_on: bool) -> void:
      if toggled_on==true:
            Global.savememory=true
            lowmode.visible=true
            $CanvasLayer/Cards.visible=false
      else:
            Global.savememory=false
            lowmode.visible=false
            $CanvasLayer/Cards.visible=true
func _on_difficulty_pressed() -> void:
      $AudioSFX.play()
      Global.difficulty = 1 if Global.difficulty == 2 else 2
      difficulty_button.text = languages[current_language]["Difficulty"]+
str(Global.difficulty) + ")"
      Global.save_settings()
func _on_movement_mode_pressed() -> void:
      $AudioSFX.play()
      Global.movement = 0 if Global.movement == 1 else 1
      if Global.movement == 0:
            movement_mode_button.text = languages[current_language]["MovementMode"]
      else:
            movement_mode_button.text = languages[current_language]["MovementMode2"]
```

```
	Global.save_settings()
	# Only free if instance exists and hasn't been freed
	if flashing_areas_instance and is_instance_valid(flashing_areas_instance):
		flashing_areas_instance.queue_free()
		flashing_areas_instance = null  # Clear reference after freeing
	if Global.difficulty == 1:
		flashing_areas_instance = flashing_areas_scene.instantiate()
	elif Global.difficulty == 2:
		flashing_areas_instance = flashing_areas_scene_2.instantiate()
	if flashing_areas_instance:
		add_child(flashing_areas_instance)
func _on_screen_limits_pressed() -> void:
	$AudioSFX.play()
	Global.screen_limits = !Global.screen_limits
	if Global.screen_limits:
		screen_limits_button.text = languages[current_language]["ScreenLimits2"]
	else:
		screen_limits_button.text = languages[current_language]["ScreenLimits"]
	Global.save_settings()
	# Only free if instance exists and hasn't been freed
	if flashing_areas_instance and is_instance_valid(flashing_areas_instance):
		flashing_areas_instance.queue_free()
		flashing_areas_instance = null  # Clear reference after freeing
	if Global.screen_limits:
		if Global.difficulty == 1:
			flashing_areas_instance = flashing_areas_scene.instantiate()
		elif Global.difficulty == 2:
			flashing_areas_instance = flashing_areas_scene_2.instantiate()
		if flashing_areas_instance:
			add_child(flashing_areas_instance)
func _on_win_condition_value_changed(value: float) -> void:
	Global.target_count = int(value)
	win_condition_label.text = languages[current_language]["WinconditionLabel"]  +
str(Global.target_count)
	Global.save_settings()
func _on_close_button_pressed():
	# Play sound if you want
	$AudioSFX.play()
	# Hide animation
	var tween_hide = create_tween()
	tween_hide.tween_property(settings_panel, "modulate", Color(1, 1, 1, 0), 0.2)
	tween_hide.tween_property(settings_panel, "scale", Vector2(0.9, 0.9), 0.2)
	tween_hide.tween_callback(func(): settings_panel.visible = false)
```

kidsgame.gd:

```
extends Node3D
@onready var color_letter_spawner = $Spawner  # Reference to the color/letter spawner
node
@onready var shape_spawner = $MeshSpawner  # Reference to the shape spawner node
@onready var custom_data_spawner = $CustomDataSpawner  # Reference to the custom data
spawner node
@onready var voice_player = $Spawner/AudioStreamPlayer
@onready var start_delay_timer = $StartDelayTimer  # Reference to the timer node
@onready var printlabel: Label = $Score2
#enum AdType {
	#NONE = 0,
	#INTERSTITIAL = 3,
```

<<OwlyFly V1.0>>

```
	#BANNER = 4,
	#REWARDED_VIDEO = 128,
	#MREC = 256
#}
#enum ShowStyle {
	#INTERSTITIAL = 3,
	#BANNER_BOTTOM = 8,
	#BANNER_TOP = 16,
	#REWARDED_VIDEO = 128,
	#MREC = 256
#}
var languages = {
	0: {  # English
		"Obstacles": " Obstacles: ",
		"MainMenu": " Main Menu "
	},
	1: {  # Chinese
		"Obstacles": " 障碍物: ",
		"MainMenu": " 主菜单 "
	}
}
var current_language # Default to English
var _interstitial_ad : InterstitialAd
var _ad_view : AdView
var gamemode = 0
var pressed
func _ready():
	print (Global.unit)
	var world = $WorldEnvironment
	if Global.savememory==true:
		world.environment.sky = load("res://resources/sky_procedural.tres")
	#else:
		#world.environment.sky = load("res://resources/sky_day.tres")
	current_language = Global.language
	pressed = 0
	$MarginContainer/MainMenu.text = languages[current_language]["MainMenu"]
	Global.showad=false
	Global.good_obstacles_collected=0
	if Global.is_premium_user==false:
		if OS.get_name() == "Android":
			MobileAds.initialize()
			var request_configuration := RequestConfiguration.new()
	request_configuration.tag_for_child_directed_treatment =
RequestConfiguration.TagForChildDirectedTreatment.TRUE
	MobileAds.set_request_configuration(request_configuration)
			printlabel.text += "iniciando..." + "\n"  # Append the message to the
label
			if _ad_view == null:
				_create_ad_view()
				printlabel.text += "loading banner..." + "\n"
			var ad_request := AdRequest.new()
			_ad_view.load_ad(ad_request)
			if _interstitial_ad:
				_interstitial_ad.destroy()
				_interstitial_ad = null
			_on_load_intersicial()
```

<<OwlyFly V1.0>>

```
	Global.victory=false
	$sfx.volume_db = linear_to_db(Global.voice_volume / 10.0)
	var startsound = preload("res://audio/effects/arcade-countdown-7007.mp3")
	$sfx.stream=startsound
	$sfx.play()
	# Pause all spawners initially
	color_letter_spawner.is_paused = true
	shape_spawner.is_paused = true
	custom_data_spawner.is_paused = true
	if Global.game_mode == "colors" or Global.game_mode == "letters":
		voice_player = $Spawner/AudioStreamPlayer
		shape_spawner.queue_free()  # Disable the shape spawner
		custom_data_spawner.queue_free()  # Disable the custom data spawner
		gamemode = 0
	elif Global.game_mode == "shapes":
		voice_player = $MeshSpawner/AudioStreamPlayer
		color_letter_spawner.queue_free()  # Disable the color/letter spawner
		custom_data_spawner.queue_free()  # Disable the custom data spawner
		gamemode = 1
	elif Global.game_mode == "custom":
		voice_player = $CustomDataSpawner/AudioStreamPlayer
		color_letter_spawner.queue_free()  # Disable the color/letter spawner
		shape_spawner.queue_free()  # Disable the shape spawner
		gamemode = 2
	elif Global.game_mode == "smallunit2dif1":
		voice_player = $CustomDataSpawner/AudioStreamPlayer
		color_letter_spawner.queue_free()  # Disable the color/letter spawner
		shape_spawner.queue_free()  # Disable the shape spawner
		gamemode = 2
	Global.inmenu = false
	$music.volume_db = linear_to_db(Global.music_volume / 10.0)
	$music.play()
	voice_player.volume_db = linear_to_db(Global.voice_volume / 10.0)
	# Start the delay timer
	start_delay_timer.wait_time = 4  # Delay of 4 seconds
	start_delay_timer.one_shot = true
	start_delay_timer.start()
func _on_start_delay_timer_timeout() -> void:
	# Enable the appropriate spawner after the delay
	if gamemode == 0:
		color_letter_spawner.is_paused = false
	elif gamemode == 1:
		shape_spawner.is_paused = false
	elif gamemode == 2:
		custom_data_spawner.is_paused = false
func _process(_delta: float) -> void:
	$Score.text= languages[current_language]["Obstacles"] +
str(Global.good_obstacles_collected) + "/" + str(Global.target_count)
	if Global.good_obstacles_collected >= Global.target_count:
		Global.good_obstacles_collected=0
		Global.victory=true
		$sfx.volume_db = Global.voice_volume
		var finishsound = preload("res://audio/effects/winning-218995.mp3")
		$sfx.stream=finishsound
		$sfx.play()
		var champion_scene = preload("res://scenes/victory_screen.tscn")
```

```
            var champion_instance = champion_scene.instantiate()
            add_child(champion_instance)
            if gamemode == 0:
                  color_letter_spawner.queue_free()  # Disable the color/letter spawner
            elif gamemode == 1:
                  shape_spawner.queue_free()  # Disable the shape spawner
            elif gamemode == 2:
                  custom_data_spawner.queue_free()  # Disable the custom data spawner
      if Global.showad:
            if Global.is_premium_user == false:
                  if Global.region == "NoChina" or Global.region=="China":
                        # Existing AdMob handling
                        destroy_ad_view()
                        if _interstitial_ad:
                              _interstitial_ad.show()
                  #elif Global.region == "China":
                        #if Engine.has_singleton("GodotAppodeal"):
                              #appodeal.hideBanner()
                              #await get_tree().create_timer(0.5).timeout
                              #show_interstitial()
            Global.showad = false
            Global.victory = false
            print (Global.unit)
            if Global.unit[2]=="main":
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
            else:
                  match Global.unit[2]:
                        "baby":
get_tree().change_scene_to_file("res://scenes/class_baby.tscn")
                        "small":
get_tree().change_scene_to_file("res://scenes/class_small.tscn")
                        "middle":
get_tree().change_scene_to_file("res://scenes/class_middle.tscn")
                        "big":
get_tree().change_scene_to_file("res://scenes/class_big.tscn")
                        "demo":
get_tree().change_scene_to_file("res://scenes/demobaby.tscn")
                        "extra1":
get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
                        "summercamp":
get_tree().change_scene_to_file("res://scenes/class_camps.tscn")
                        _: get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_main_menu_pressed() -> void:
      if pressed == 0:
            pressed = 1
            Global.good_obstacles_collected = 0
            if Global.is_premium_user == false:
                  if Global.region == "NoChina" or Global.region=="China":
                        # Existing AdMob flow (unchanged)
                        destroy_ad_view()
                        if _interstitial_ad:
                              _interstitial_ad.show()
            # Always change scene immediately (like Godot 3.6)
            Global.showad = false
            Global.victory = false
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
```

<<OwlyFly V1.0>>

```
func _create_ad_view() -> void:
	#free memory
	if _ad_view:
		destroy_ad_view()
	var unit_id : String
	if OS.get_name() == "Android":
		unit_id = "ca-app-pub-3940256099942544~3347511713"
		printlabel.text += "asign id" + "\n"
	elif OS.get_name() == "iOS":
		unit_id = "ca-app-pub-3940256099942544/6300978111"
	#_ad_view = AdView.new(unit_id, AdSize.BANNER, AdPosition.Values.TOP) #CRITICAL
here we deactivate banners
	#printlabel.text += "ad loaded" + "\n"
func destroy_ad_view() -> void:
	if _ad_view:
		#always call this method on all AdFormats to free memory on Android/iOS
		_ad_view.destroy()
		_ad_view = null
func _on_load_intersicial():
	#free memory
	if _interstitial_ad:
		#always call this method on all AdFormats to free memory on Android/iOS
		_interstitial_ad.destroy()
		_interstitial_ad = null
	var unit_id : String
	if OS.get_name() == "Android":
		unit_id = " ca-app-pub-3940256099942544~3347511713"
	elif OS.get_name() == "iOS":
		unit_id = " ca-app-pub-3940256099942544/6300978111"
	var interstitial_ad_load_callback := InterstitialAdLoadCallback.new()
	interstitial_ad_load_callback.on_ad_failed_to_load = func(adError : LoadAdError) ->
void:
		print(adError.message)
	interstitial_ad_load_callback.on_ad_loaded = func(interstitial_ad : InterstitialAd)
-> void:
		print("interstitial ad loaded" + str(interstitial_ad._uid))
		_interstitial_ad = interstitial_ad
	InterstitialAdLoader.new().load(unit_id, AdRequest.new(),
interstitial_ad_load_callback)
```

camera3D.gd:

```
extends Camera3D
@export var player_path: NodePath
@export var follow_distance: float = -8.0
@export var follow_height: float = 5.0
@export var follow_speed: float = 8.0
var player: Node3D
func _ready():
	player = get_node(player_path)
func _process(delta):
	if player:
		var target_position = player.global_position + player.transform.basis.z * -
follow_distance + Vector3(0, follow_height, 0)
		global_position = global_position.move_toward(target_position, follow_speed *
delta)
		look_at(player.global_position, Vector3.UP)
```

path_generator.gd:

<<OwlyFly V1.0>>

```
extends Node3D
@export var path_segment_scene: PackedScene
@export var segment_length = 10.0
@export var initial_segments = 5  # Load 5 initial segments
@export var min_distance_between_segments = 120.0  # Minimum distance between segments
@export var removal_offset = 100.0  # Distance behind to remove segments
var segments = []
func _ready():
	match Global.scene:
		0:  # Magic Forest
			min_distance_between_segments = 120.0
		1:  # City
			min_distance_between_segments = 170.0
		2:  # Pirate
			min_distance_between_segments = 700.0
			initial_segments = 3
		3:  # Cathedral
			min_distance_between_segments = 930.0
			initial_segments = 4
		4:  # City Tunnel
			min_distance_between_segments = 2300.0
			initial_segments = 3
		5:  # Winter
			min_distance_between_segments = 380.0
			initial_segments = 5
		_:
			min_distance_between_segments = 120.0
	if Global.savememory==true:
		min_distance_between_segments = 135.0
		initial_segments = 5
	# Load the first 5 segments initially
	for i in range(initial_segments):
		add_segment()
func _process(delta):
	var global_speed = Global.game_speed
	update_segments(delta, global_speed)
	ensure_continuity()
func update_segments(delta, global_speed):
	var segments_to_remove = []
	for segment in segments:
		segment.global_position.z += global_speed * delta
		if segment.global_position.z > segment_length * (initial_segments - 1) +
removal_offset:
			segments_to_remove.append(segment)
	for segment in segments_to_remove:
		segment.queue_free()
		segments.erase(segment)
func add_segment():
	var last_segment_position = Vector3()
	if segments.size() > 0:
		last_segment_position = segments[segments.size() - 1].global_position
	var new_segment = path_segment_scene.instantiate()
	new_segment.global_position = last_segment_position + Vector3(0, 0, -segment_length
- min_distance_between_segments)
	add_child(new_segment)
	segments.append(new_segment)
```

<<OwlyFly V1.0>>

```
func ensure_continuity():
	while segments.size() < initial_segments:
		add_segment()
```

spawner.gd:

```
extends Node3D
@export var player: CharacterBody3D
@export var good_obstacle_scene: PackedScene
@export var bad_obstacle_scene: PackedScene
@export var flashing_areas_scene : PackedScene
@export var flashing_areas_scene_2 : PackedScene
@export var grid_spacing = Vector3(17, 17, 0)  # Adjusted grid spacing
@export var obstacle_distance_ahead = 250.0  # Distance ahead of player to spawn
obstacles
@export var removal_distance = 12.0  # Distance behind the player to remove obstacles
@onready var label =  $CanvasLayer/Label # Reference to a label to display the target
item (color/letter)
@onready var audio_player = $AudioStreamPlayer  # Reference to an audio player node
@onready var announce_timer = $Timer  # Reference to a Timer node
var obstacles = []
var next_target_item
var repeat_count = 0
var is_paused = true  # Initialize paused state
var flashing_areas_instance
func _process(_delta):
	if is_paused:
		return  # Skip the rest of the function if paused
	remove_passed_obstacles()
	if should_spawn_new_obstacles():
		spawn_obstacles()
func spawn_obstacles():
	clear_obstacles()
	if Global.screen_limits==true:
		if Global.difficulty == 1:
			flashing_areas_instance = flashing_areas_scene.instantiate()
		elif Global.difficulty == 2:
			flashing_areas_instance = flashing_areas_scene_2.instantiate()
		add_child(flashing_areas_instance)
	var player_position = player.global_position
	var obstacle_position_z = player_position.z - obstacle_distance_ahead
	var grid_positions = create_grid_positions()
	var good_obstacle_index = randi() % grid_positions.size()
	if Global.game_mode == "colors":
		next_target_item = get_random_color()
		Global.target_color = next_target_item
	else:
		next_target_item = get_random_letter()
		Global.target_letter = next_target_item
	for i in range(grid_positions.size()):
		var obstacle_scene
		if i == good_obstacle_index:
			obstacle_scene = good_obstacle_scene.instantiate()
			set_obstacle_item(obstacle_scene, next_target_item)
		else:
			obstacle_scene = bad_obstacle_scene.instantiate()
			var chosen_item = get_unique_bad_item(next_target_item)
			set_obstacle_item(obstacle_scene, chosen_item)
```

<<OwlyFly V1.0>>

```
            # Set grid position properties
            var grid_pos = grid_positions[i]
            var current_column = int(grid_pos.x / grid_spacing.x)
            var current_row = int(grid_pos.y / grid_spacing.y) if Global.difficulty == 2
else 0
            obstacle_scene.grid_column = current_column
            obstacle_scene.grid_row = current_row
            obstacle_scene.global_position = grid_positions[i] + Vector3(0, 0,
obstacle_position_z)
            obstacles.append(obstacle_scene)
            add_child(obstacle_scene)
      announce_target_item()
func remove_passed_obstacles():
      var player_position = player.global_position.z
      var obstacles_to_remove = []
      for obstacle in obstacles:
            if is_instance_valid(obstacle) and obstacle.global_position.z >
player_position + removal_distance:
                  obstacles_to_remove.append(obstacle)
      for obstacle in obstacles_to_remove:
            if is_instance_valid(obstacle):
                  obstacle.queue_free()
                  obstacles.erase(obstacle)
func should_spawn_new_obstacles():
      return obstacles.size() == 0
func create_grid_positions():
      var grid_positions = []
      if Global.difficulty == 1:
            for x in range(3):
                  grid_positions.append(Vector3(x * grid_spacing.x, 0, 0))
      elif Global.difficulty == 2:
            for x in range(3):
                  for y in range(2):
                        grid_positions.append(Vector3(x * grid_spacing.x, y *
grid_spacing.y, 0))
      return grid_positions
func clear_obstacles():
      for obstacle in obstacles:
            if is_instance_valid(obstacle):
                  obstacle.queue_free()
      obstacles.clear()
func set_obstacle_item(obstacle, item):
      if Global.game_mode == "colors":
            set_obstacle_color(obstacle, item)
      else:  # letters mode
            set_obstacle_letter(obstacle, item)
func set_obstacle_color(obstacle, color: Color):
      var mesh_instance =
obstacle.get_node("Sketchfab_Scene/Sketchfab_model/MeshInstance3D")
      if mesh_instance:
            for i in range(mesh_instance.get_surface_material_count()):
                  var material = mesh_instance.get_surface_material(i)
                  if material:
                        var new_material = material.duplicate()
                        new_material.albedo_color = color
                        mesh_instance.set_surface_material(i, new_material)
```

<<OwlyFly V1.0>>

```
func set_obstacle_letter(obstacle, letter: String):
	var label2 = obstacle.get_node("Label")  # Assuming each obstacle has a Label node
	label2.text = letter
func announce_target_item():
	var audio_stream: AudioStream
	if Global.game_mode == "colors":
		var color_name = Global.color_data[Global.target_color].name
		label.text =  color_name
		audio_stream = Global.color_data[Global.target_color].audio
	else:  # letters mode
		label.text = Global.target_letter
		audio_stream = Global.letter_data[Global.target_letter].audio
	audio_player.stream = audio_stream
	for t in 3:
		audio_player.play()
		await audio_player.finished
		await get_tree().create_timer(0.3).timeout
func get_random_color() -> Color:
	var keys = Global.color_data.keys()
	return keys[randi() % keys.size()]
func get_random_letter() -> String:
	var keys = Global.letter_data.keys()
	return keys[randi() % keys.size()]
func get_unique_bad_item(good_item):
	if Global.game_mode == "colors":
		return get_unique_bad_color(good_item)
	else:
		return get_unique_bad_letter(good_item)
func get_unique_bad_color(good_color: Color) -> Color:
	var keys = Global.color_data.keys()
	var chosen_color = keys[randi() % keys.size()]
	while chosen_color == good_color:
		chosen_color = keys[randi() % keys.size()]
	return chosen_color
func get_unique_bad_letter(good_letter: String) -> String:
	var keys = Global.letter_data.keys()
	var chosen_letter = keys[randi() % keys.size()]
	while chosen_letter == good_letter:
		chosen_letter = keys[randi() % keys.size()]
	return chosen_letter
```

player.gd:

```
extends CharacterBody3D
@export var speed = 10.0
@export var grid_spacing = Vector3(17, 17, 1)  # Adjusted grid spacing
@onready var model_holder: Node3D = $ModelHolder
var target_position = Vector3()
var current_column = 1
var current_row = 0
var current_skin_instance: Node3D
var target_color = Color(0, 1, 0)  # Green
var grid_column: int
var grid_row: int = 0
func _ready():
	target_position = global_position
	adjust_grid_size()
	add_to_group("player")
```

```
	load_skin()
func load_skin():
	# Clear existing skin
	for child in model_holder.get_children():
		child.queue_free()
	# Load new skin
	var skin_scene = get_skin_scene(Global.equipped_skin)
	current_skin_instance = skin_scene.instantiate()
	model_holder.add_child(current_skin_instance)
	# Play animation if available
	var anim_player = current_skin_instance.find_child("AnimationPlayer")
	if anim_player and anim_player.get_animation_list().size() > 0:
		anim_player.play(anim_player.get_animation_list()[0])
		anim_player.speed_scale = 1.0
		if Global.equipped_skin==3: #this is the index of the red dragon
			anim_player.speed_scale = 2.0
func get_skin_scene(index: int) -> PackedScene:
	match index:
		0: return preload("res://3DModels/owl/owl.glb")
		1: return preload("res://3DModels/ww1plane/ww1_plane.glb")
		2: return preload("res://3DModels/eagle/white_eagle.glb")
		3: return preload("res://3DModels/dragon1/dragon_flying.glb")
		4: return preload("res://3DModels/bat/emerald_bat.glb")
		5: return preload("res://3DModels/dragon2/european_dragon.glb")
		_: return preload("res://3DModels/owl/owl.glb")  # Fallback
func _process(delta):
	if Global.victory==false:
		handle_movement()
	Global.current_column=current_column
	Global.current_row=current_row
	move_towards_target(delta)
func handle_movement():
	if Input.is_action_just_pressed("ui_left") and current_column > 0:
		current_column -= 1
	elif Input.is_action_just_pressed("ui_right") and current_column < 2:
		current_column += 1
	if Global.difficulty == 2:
		if Input.is_action_just_pressed("ui_up") and current_row < 1:
			current_row += 1
		elif Input.is_action_just_pressed("ui_down") and current_row > 0:
			current_row -= 1
	# Handle mouse input
	if Global.movement==0:
		if Input.is_mouse_button_pressed(MOUSE_BUTTON_LEFT):
			var click_position = get_viewport().get_mouse_position()
			var screen_width = get_viewport().size.x
			var screen_height = get_viewport().size.y
			# Determine the clicked column
			if click_position.x < screen_width / 3:
				current_column = 0  # Move to the left column
			elif click_position.x < 2 * screen_width / 3:
				current_column = 1  # Move to the center column
			else:
				current_column = 2  # Move to the right column
			# Determine the clicked row (only for difficulty 2)
			if Global.difficulty == 2:
```

```
                        if click_position.y < screen_height / 2:
                              current_row = 1  # Move to the top row
                        else:
                              current_row = 0  # Move to the bottom row
      target_position = Vector3(current_column * grid_spacing.x, current_row *
grid_spacing.y, global_position.z)
func move_towards_target(delta):
      global_position = global_position.move_toward(target_position, speed * delta)
func adjust_grid_size():
      if Global.difficulty == 1:
            current_row = 0
      else:
            current_row = 0
func _on_camera_2d_swipedown() -> void:
      if Global.difficulty==2:
            if current_row > 0:
                  current_row -= 1
                  target_position = Vector3(current_column * grid_spacing.x, current_row
* grid_spacing.y, global_position.z)
      Global.current_column=current_column
      Global.current_row=current_row
func _on_camera_2d_swipeleft() -> void:
      if current_column > 0:
            current_column -= 1
            target_position = Vector3(current_column * grid_spacing.x, current_row *
grid_spacing.y, global_position.z)
      Global.current_column=current_column
      Global.current_row=current_row
func _on_camera_2d_swiperight() -> void:
      if current_column < 2:
            current_column += 1
            target_position = Vector3(current_column * grid_spacing.x, current_row *
grid_spacing.y, global_position.z)
      Global.current_column=current_column
      Global.current_row=current_row
func _on_camera_2d_swipeup() -> void:
      if Global.difficulty ==2:
            if current_row < 1:
                  current_row += 1
                  target_position = Vector3(current_column * grid_spacing.x, current_row
* grid_spacing.y, global_position.z)
      Global.current_column=current_column
      Global.current_row=current_row
```

meshspawner.gd:

```
extends Node3D
@export var player: CharacterBody3D
@export var good_obstacle_scene: PackedScene
@export var bad_obstacle_scene: PackedScene
@export var flashing_areas_scene : PackedScene
@export var flashing_areas_scene_2 : PackedScene
@export var grid_spacing = Vector3(17, 17, 0)
@export var obstacle_distance_ahead = 250.0
@export var removal_distance = 12.0
@onready var label = $CanvasLayer/Label
@onready var audio_player = $AudioStreamPlayer
@onready var announce_timer = $Timer
```

<<OwlyFly V1.0>>

```
var obstacles = []
var next_target_shape: String
var repeat_count = 0
var is_paused = true
var flashing_areas_instance
#func _ready():
	#spawn_obstacles()
func _process(_delta):
	if is_paused:
		return  # Skip the rest of the function if paused
	remove_passed_obstacles()
	if should_spawn_new_obstacles():
		spawn_obstacles()
func spawn_obstacles():
	clear_obstacles()
	if Global.screen_limits==true:
		if Global.difficulty == 1:
			flashing_areas_instance = flashing_areas_scene.instantiate()
		elif Global.difficulty == 2:
			flashing_areas_instance = flashing_areas_scene_2.instantiate()
		add_child(flashing_areas_instance)
	var player_position = player.global_position
	var obstacle_position_z = player_position.z - obstacle_distance_ahead
	var grid_positions = create_grid_positions()
	var good_obstacle_index = randi() % grid_positions.size()
	next_target_shape = get_random_shape()
	Global.target_shape = next_target_shape
	# Place the good obstacle
	var good_obstacle = good_obstacle_scene.instantiate()
	set_obstacle_item(good_obstacle, next_target_shape)
	# Set grid position for good obstacle
	var good_grid_pos = grid_positions[good_obstacle_index]
	good_obstacle.grid_column = int(good_grid_pos.x / grid_spacing.x)
	good_obstacle.grid_row = int(good_grid_pos.y / grid_spacing.y) if Global.difficulty
== 2 else 0
	good_obstacle.global_position = good_grid_pos + Vector3(0, 0, obstacle_position_z)
	obstacles.append(good_obstacle)
	add_child(good_obstacle)
	# Place bad obstacles
	for i in range(grid_positions.size()):
		if i == good_obstacle_index:
			continue
		var bad_obstacle = bad_obstacle_scene.instantiate()
		var chosen_item = get_unique_bad_item(next_target_shape)
		set_obstacle_item(bad_obstacle, chosen_item)
		# Set grid position for bad obstacles
		var grid_pos = grid_positions[i]
		bad_obstacle.grid_column = int(grid_pos.x / grid_spacing.x)
		bad_obstacle.grid_row = int(grid_pos.y / grid_spacing.y) if Global.difficulty
== 2 else 0
		bad_obstacle.global_position = grid_pos + Vector3(0, 0, obstacle_position_z)
		obstacles.append(bad_obstacle)
		add_child(bad_obstacle)
	announce_target_item()
func remove_passed_obstacles():
	var player_position = player.global_position.z
```

<<OwlyFly V1.0>>

```
	var obstacles_to_remove = []
	for obstacle in obstacles:
		if is_instance_valid(obstacle) and obstacle.global_position.z >
player_position + removal_distance:
			obstacles_to_remove.append(obstacle)
	for obstacle in obstacles_to_remove:
		if is_instance_valid(obstacle):
			obstacle.queue_free()
			obstacles.erase(obstacle)
func should_spawn_new_obstacles():
	return obstacles.size() == 0
func create_grid_positions():
	var grid_positions = []
	if Global.difficulty == 1:
		for x in range(3):
			grid_positions.append(Vector3(x * grid_spacing.x, 0, 0))
	elif Global.difficulty == 2:
		for x in range(3):
			for y in range(2):
				grid_positions.append(Vector3(x * grid_spacing.x, y *
grid_spacing.y, 0))
	return grid_positions
func clear_obstacles():
	for obstacle in obstacles:
		if is_instance_valid(obstacle):
			obstacle.queue_free()
	obstacles.clear()
func set_obstacle_item(obstacle, item):
	var quad_mesh_instance = obstacle.get_node("MeshInstance3D")
	var material = StandardMaterial3D.new()
	material.albedo_texture = Global.shape_data[item].image
	quad_mesh_instance.material_override = material
func announce_target_item():
	var audio_stream = Global.shape_data[Global.target_shape].audio
	label.text = Global.target_shape
	audio_player.stream = audio_stream
	for t in 3:
		audio_player.volume_db=Global.voice_volume
		audio_player.play()
		await audio_player.finished
		await get_tree().create_timer(0.3).timeout
func get_random_shape() -> String:
	var keys = Global.shape_data.keys()
	return keys[randi() % keys.size()]
func get_unique_bad_item(good_shape):
	var keys = Global.shape_data.keys()
	var chosen_shape
	while true:
		chosen_shape = keys[randi() % keys.size()]
		if chosen_shape != good_shape:
			break
	return chosen_shape
```

custom_data_spawner.gd:

```
extends Node3D
@export var player: CharacterBody3D
@export var good_obstacle_scene: PackedScene
```

<<OwlyFly V1.0>>

```
@export var bad_obstacle_scene: PackedScene
@export var flashing_areas_scene : PackedScene
@export var flashing_areas_scene_2 : PackedScene
@export var grid_spacing = Vector3(17, 17, 0)
@export var obstacle_distance_ahead = 250.0
@export var removal_distance = 12.0
@onready var label = $CanvasLayer/Label
@onready var audio_player = $AudioStreamPlayer
@onready var announce_timer = $Timer
var obstacles = []
var next_target_item: String
var custom_topic = {}
var is_paused = true
var flashing_areas_instance
func _ready():
      if Global.game_mode == "custom" and Global.current_topic in Global.custom_topics:
            custom_topic = Global.custom_topics[Global.current_topic]
            if custom_topic.size() > 0:
                  print("all ready")
            else:
                  print("Custom topic is empty: " + Global.current_topic)
      else:
            print("Custom topic not found")
func _process(_delta):
      if is_paused:
            return  # Skip the rest of the function if paused
      remove_passed_obstacles()
      if should_spawn_new_obstacles():
            spawn_obstacles()
func spawn_obstacles():
      clear_obstacles()
      if Global.screen_limits==true:
            if Global.difficulty == 1:
                  flashing_areas_instance = flashing_areas_scene.instantiate()
            elif Global.difficulty == 2:
                  flashing_areas_instance = flashing_areas_scene_2.instantiate()
            add_child(flashing_areas_instance)
      var player_position = player.global_position
      var obstacle_position_z = player_position.z - obstacle_distance_ahead
      var grid_positions = create_grid_positions()
      var good_obstacle_index = randi() % grid_positions.size()
      next_target_item = get_random_custom_item()
      Global.target_shape = next_target_item
      # Place the good obstacle
      var good_obstacle = good_obstacle_scene.instantiate()
      set_obstacle_item(good_obstacle, next_target_item)
      # Set grid position for good obstacle
      var good_grid_pos = grid_positions[good_obstacle_index]
      good_obstacle.grid_column = int(good_grid_pos.x / grid_spacing.x)
      good_obstacle.grid_row = int(good_grid_pos.y / grid_spacing.y) if Global.difficulty
== 2 else 0
      good_obstacle.global_position = good_grid_pos + Vector3(0, 0, obstacle_position_z)
      obstacles.append(good_obstacle)
      add_child(good_obstacle)
      # Place bad obstacles
      for i in range(grid_positions.size()):
```

<<OwlyFly V1.0>>

```
            if i == good_obstacle_index:
                  continue
            var bad_obstacle = bad_obstacle_scene.instantiate()
            var chosen_item = get_unique_bad_item(next_target_item)
            set_obstacle_item(bad_obstacle, chosen_item)
            # Set grid position for bad obstacles
            var grid_pos = grid_positions[i]
            bad_obstacle.grid_column = int(grid_pos.x / grid_spacing.x)
            bad_obstacle.grid_row = int(grid_pos.y / grid_spacing.y) if Global.difficulty
== 2 else 0
            bad_obstacle.global_position = grid_pos + Vector3(0, 0, obstacle_position_z)
            obstacles.append(bad_obstacle)
            add_child(bad_obstacle)
      announce_target_item()
func remove_passed_obstacles():
      var player_position = player.global_position.z
      var obstacles_to_remove = []
      for obstacle in obstacles:
            if is_instance_valid(obstacle) and obstacle.global_position.z >
player_position + removal_distance:
                  obstacles_to_remove.append(obstacle)
      for obstacle in obstacles_to_remove:
            if is_instance_valid(obstacle):
                  obstacle.queue_free()
                  obstacles.erase(obstacle)
func should_spawn_new_obstacles():
      return obstacles.size() == 0
func create_grid_positions():
      var grid_positions = []
      if Global.difficulty == 1:
            for x in range(3):
                  grid_positions.append(Vector3(x * grid_spacing.x, 0, 0))
      elif Global.difficulty == 2:
            for x in range(3):
                  for y in range(2):
                        grid_positions.append(Vector3(x * grid_spacing.x, y *
grid_spacing.y, 0))
      return grid_positions
func clear_obstacles():
      for obstacle in obstacles:
            if is_instance_valid(obstacle):
                  obstacle.queue_free()
      obstacles.clear()
func set_obstacle_item(obstacle, item):
      var quad_mesh_instance = obstacle.get_node("MeshInstance3D")
      var material = StandardMaterial3D.new()
      if custom_topic.has(item):
            material.albedo_texture = custom_topic[item]["image"]
      else:
            print("Item not found in custom_topic: " + item)
      quad_mesh_instance.material_override = material
func announce_target_item():
      if custom_topic.has(Global.target_shape):
            var audio_stream = custom_topic[Global.target_shape]["audio"]
            if audio_stream == null:
                  print("Audio stream is null for item: " + Global.target_shape)
```

```
            else:
                  label.text =  Global.target_shape
                  audio_player.stream = audio_stream
                  var original_length = audio_stream.get_length()
                  var loop_count = 3 if original_length <= 1.5 else 1
                  var audio_length = min(original_length, 3.0)
                  for i in range(loop_count):
                        audio_player.play()
                        await get_tree().create_timer(audio_length).timeout
                        audio_player.stop()
                        if i < loop_count - 1:  # Only add pause between iterations
                              await get_tree().create_timer(0.3).timeout
      else:
            print("Target shape not found in custom_topic: " + Global.target_shape)
func get_random_custom_item() -> String:
      var keys = custom_topic.keys()
      if keys.size() == 0:
            print("No items in custom topic")
            return ""
      return keys[randi() % keys.size()]
func get_unique_bad_item(good_item):
      var keys = custom_topic.keys()
      if keys.size() == 0:
            print("No items in custom topic")
            return ""
      var chosen_item
      while true:
            chosen_item = keys[randi() % keys.size()]
            if chosen_item != good_item:
                  break
      return chosen_item
extralevels.gd:
extends Control
var languages = {
      0: {  # English
            "Unit1": " Places ",
            "Unit2": " Jobs ",
            "Unit3": " Family ",
            "Unit4": " Toys ",
            "Unit5": " Pets ",
            "Unit6": "      Transportation      ",
            "Review": " Review ",
            "Words": " Words ",
            "Sentences": " Sentences ",
            "MainMenu": " Main Menu ",
            "Words1v1": " 1v1 Words ",
            "Sentences1v1": " 1v1 Sentences ",
      },
      1: {  # Chinese
            "Unit1": " 地方 ",
            "Unit2": " 职业 ",
            "Unit3": " 家庭 ",
            "Unit4": " 玩具 ",
            "Unit5": " 宠物 ",
            "Unit6": " 交通 ",
```

<<OwlyFly V1.0>>

```
            "Review": " 复习 ",
            "Words": " 单词 ",
            "Sentences": " 句子 ",
            "MainMenu": "      主菜单      ",
            "Words1v1": " 1v1 单词  ",
            "Sentences1v1": " 1v1 句子 ",
      }
}
const UNIT_OPTIONS = preload("res://scenes/unit_options_panel.tscn")
var current_language # Default to English
var pressed
func _ready() -> void:
      Global.inmenu=true
      Global.unit=["extra1"]
      print(Global.unit)
      current_language = Global.language
      $AudioSFX.volume_db = Global.voice_volume
      pressed = 0
      $MarginContainer/VBoxContainer/MainMenu.text =
languages[current_language]["MainMenu"]
      $MarginContainer/VBoxContainer/Unit1.text = languages[current_language]["Unit1"]
      $MarginContainer/VBoxContainer/Unit2.text = languages[current_language]["Unit2"]
      $MarginContainer/VBoxContainer/Unit3.text = languages[current_language]["Unit3"]
      $MarginContainer/VBoxContainer/Unit4.text = languages[current_language]["Unit4"]
      $MarginContainer/VBoxContainer/Unit5.text = languages[current_language]["Unit5"]
      $MarginContainer/VBoxContainer/Unit6.text = languages[current_language]["Unit6"]
func _process_topic_resource(topic_data: Resource) -> Dictionary:
      var result = {}
      for entry in topic_data.entries:
            result[entry.name] = {
                  "name": entry.name,
                  "image": load(entry.image_path),
                  "audio": load(entry.audio_path)
            }
      return result
func _on_main_menu_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_unit_1_pressed() -> void:
      $AudioSFX.play()
      var topic_data = load("res://topics/smallunit1dif1.tres")
      var babyunit1dif1 = _process_topic_resource(topic_data)
      var topic_data2 = load("res://topics/smallunit1dif2.tres")
      var babyunit1dif2 = _process_topic_resource(topic_data2)
      var videos
      if Global.region=="China":
            videos = {
                  "Where are you going? Song": {"platform": "bilibili", "id":
"BV1mpg5zFEtr"},
                  }
      else:
            videos = {
```

```
                  "Where are you going? Song": {"platform": "youtube", "id":
"krPMYK2aY3o"},
                  }
      await $AudioSFX.finished
      var unit = [babyunit1dif1,
babyunit1dif2,"extra1",load("res://images/aimier/questions/whereareyougoing.mp3"),
videos]
      Global.unit=unit
      var options_panel = UNIT_OPTIONS.instantiate()
      add_child(options_panel)
func _on_unit_2_pressed() -> void:
      $AudioSFX.play()
      var videos
      if Global.region=="China":
            videos = {
                  "Job Song": {"platform": "bilibili", "id": "BV1uGufzQE8F"},
                  }
      else:
            videos = {
                  "Job Song": {"platform": "youtube", "id": "2nesqKP9-5c"},
                  }
      var topic_data = load("res://topics/middleunit2dif1.tres")
      var babyunit2dif1 = _process_topic_resource(topic_data)
      var topic_data2 = load("res://topics/middleunit2dif2.tres")
      var babyunit2dif2 = _process_topic_resource(topic_data2)
      await $AudioSFX.finished
      var unit = [babyunit2dif1,
babyunit2dif2,"extra1",load("res://images/aimier/questions/whoishewhoisshe.mp3"),videos]
      Global.unit=unit
      var options_panel = UNIT_OPTIONS.instantiate()
      add_child(options_panel)
func _on_unit_3_pressed() -> void:
      $AudioSFX.play()
      var topic_data = load("res://topics/babyunit3dif1.tres")
      var babyunit3dif1 = _process_topic_resource(topic_data)
      var topic_data2 = load("res://topics/babyunit3dif2.tres")
      var babyunit3dif2 = _process_topic_resource(topic_data2)
      var videos
      if Global.region=="China":
            videos = {
                  "Baby Shark Song": {"platform": "bilibili", "id": "BV1LGgTz9EkT"},
                  }
      else:
            videos = {
                  "Baby Shark Song": {"platform": "youtube", "id": "XqZsoesa55w"},
                  }
      await $AudioSFX.finished
      var unit = [babyunit3dif1, babyunit3dif2,"extra1",
load("res://images/aimier/questions/whosthis.mp3"), videos]
      Global.unit=unit
      var options_panel = UNIT_OPTIONS.instantiate()
      add_child(options_panel)
func _on_unit_4_pressed() -> void:
      $AudioSFX.play()
      var topic_data = load("res://topics/babyunit1dif1.tres")
      var babyunit1dif1 = _process_topic_resource(topic_data)
```

```
	var topic_data2 = load("res://topics/babyunit1dif2.tres")
	var babyunit1dif2 = _process_topic_resource(topic_data2)
	var videos
	if Global.region=="China":
		videos = {
			"The Toys Song": {"platform": "bilibili", "id": "BV1hFunzjEzw"},
			}
	else:
		videos = {
			"The Toys Song": {"platform": "youtube", "id": "8-SWzpdcl6E"},
			}
	await $AudioSFX.finished
	var unit = [babyunit1dif1,
babyunit1dif2,"extra1",load("res://images/aimier/questions/whatisit.mp3"), videos]
	Global.unit=unit
	var options_panel = UNIT_OPTIONS.instantiate()
	add_child(options_panel)
func _on_unit_5_pressed() -> void:
	$AudioSFX.play()
	var topic_data = load("res://topics/middleunit5dif1.tres")
	var middleunit5dif1 = _process_topic_resource(topic_data)
	var topic_data2 = load("res://topics/middleunit5dif2.tres")
	var middleunit5dif2 = _process_topic_resource(topic_data2)
	var videos
	if Global.region=="China":
		videos = {
			"Pets Song": {"platform": "bilibili", "id": "BV1bGufzQEPp"},
			}
	else:
		videos = {
			"Pets Song": {"platform": "youtube", "id": "pWepfJ-8XU0"},
			}
	await $AudioSFX.finished
	var unit = [middleunit5dif1,
middleunit5dif2,"extra1",load("res://images/aimier/questions/whatisthis.mp3"), videos]
	Global.unit=unit
	var options_panel = UNIT_OPTIONS.instantiate()
	add_child(options_panel)
func _on_unit_6_pressed() -> void:
	$AudioSFX.play()
	var topic_data = load("res://topics/smallunit2dif1.tres")
	var smallunit2dif1 = _process_topic_resource(topic_data)
	var topic_data2 = load("res://topics/smallunit2dif2.tres")
	var smallunit2dif2 = _process_topic_resource(topic_data2)
	var videos
	if Global.region=="China":
		videos = {
			"Transportation song": {"platform": "bilibili", "id": "BV18RuSzwEx5"},
			}
	else:
		videos = {
			"Transportation song": {"platform": "youtube", "id": "SWSbejC47hk"},
			}
	await $AudioSFX.finished
	var unit = [smallunit2dif1,
smallunit2dif2,"extra1",load("res://images/aimier/questions/whatdoyousee.mp3"),videos]
```

<<OwlyFly V1.0>>

```
      Global.unit=unit
      var options_panel = UNIT_OPTIONS.instantiate()
      add_child(options_panel)
```

unit_options_panel.gd:

```
extends PanelContainer
var data
var pressed
var languages = {
      0: {  # English
            "Review": " Review ",
            "Words": " Words ",
            "Sentences": " Sentences ",
            "Close": "  Close  "
      },
      1: {  # Chinese
            "Review": " 复习 ",
            "Words": " 单词 ",
            "Sentences": " 句子 ",
            "Close": "  关闭  "
      }
}
# Called when the node enters the scene tree for the first time.
func _ready() -> void:
      data=Global.unit
      pressed = 0
      var lang = Global.language
      # Set button texts
      $OptionsUnit1/ReviewButton.text = languages[lang]["Review"]
      $OptionsUnit1/Row1/OwlyWordsButton.text = languages[lang]["Words"]
      $OptionsUnit1/Row1/OwlySentencesButton.text = languages[lang]["Sentences"]
      $"OptionsUnit1/Row2/1v1WordsButton".text = languages[lang]["Words"]
      $"OptionsUnit1/Row2/1v1SentencesButton".text = languages[lang]["Sentences"]
      $OptionsUnit1/Row3/WamWordsButton.text = languages[lang]["Words"]
      $OptionsUnit1/Row3/WamSentencesButton.text = languages[lang]["Sentences"]
      $OptionsUnit1/CloseButton.text = languages[lang]["Close"]
      scale = Vector2(0.8, 0.8)
      var tween = create_tween()
      tween.tween_property(self, "scale", Vector2(1, 1), 0.8)\
            .set_ease(Tween.EASE_OUT).set_trans(Tween.TRANS_BACK)
func _on_review_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            #var unit = [data[0], data[1],data[2],data[3], data[4]]
      get_tree().change_scene_to_file("res://scenes/topic_viewer.tscn")
func _on_owly_words_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[0]
      get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_owly_sentences_button_pressed() -> void:
      if pressed == 0:
```

```
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[1]
      get_tree().change_scene_to_file("res://scenes/choose_landscape.tscn")
func _on_v_1_words_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[0]
            get_tree().change_scene_to_file("res://scenes/setup_1v_1.tscn")
func _on_v_1_sentences_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[1]
            get_tree().change_scene_to_file("res://scenes/setup_1v_1.tscn")
func _on_wam_words_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[0]
      get_tree().change_scene_to_file("res://scenes/whackamole_setup.tscn")
func _on_wam_sentences_button_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            Global.game_mode = "custom"
            Global.custom_topics[Global.current_topic] = data[1]
      get_tree().change_scene_to_file("res://scenes/whackamole_setup.tscn")
func _on_close_button_pressed() -> void:
      queue_free()
```

topic_viewer.gd:

```
extends Control
@onready var scroll_container = $CanvasLayer/ScrollContainer
@onready var vbox = $CanvasLayer/ScrollContainer/VBoxContainer
@onready var scroll_container_2: ScrollContainer = $CanvasLayer/ScrollContainer2
@onready var question: Button = $CanvasLayer/VBoxContainer2/Question
# New references for overlay
@onready var overlay_panel = $CanvasLayer/OverlayPanel
@onready var overlay_scroll = $CanvasLayer/OverlayPanel/OverlayScroll
@onready var overlay_vbox = $CanvasLayer/OverlayPanel/OverlayScroll/OverlayVBox
@onready var close_button = $CanvasLayer/OverlayPanel/Close
@onready var multimedia: Button = $CanvasLayer/VBoxContainer2/Multimedia
var pressed = 0
var current_topic: Dictionary
var current_topic2: Dictionary
var videos: Dictionary = {}
```

<<OwlyFly V1.0>>

```
var languages = {
	0: {  # 英语 English
		"MainMenu": " Main Menu ",
		"GoBack": "Go Back",
		"Words": "Words",
		"Sentences": "Sentences"
	},
	1: {  # 中文 Chinese
		"MainMenu": " 主菜单 ",
		"GoBack": "返回",
		"Words": "单词",
		"Sentences": "句子"
	}
}
var current_language = Global.language
func _ready():
	Global.inmenu=false
	overlay_panel.visible = false
	# Add semi-transparent gray background
	var style = StyleBoxFlat.new()
	style.bg_color = Color(0.3, 0.3, 0.3, 0.8)  # Gray with 30% opacity
	style.set_content_margin_all(10)  # Add some padding
	style.corner_radius_top_left = 10
	style.corner_radius_top_right = 10
	style.corner_radius_bottom_left = 10
	style.corner_radius_bottom_right = 10
	scroll_container.add_theme_stylebox_override("panel", style)
	scroll_container_2.add_theme_stylebox_override("panel", style)
	overlay_scroll.add_theme_stylebox_override("panel", style)
	current_topic=Global.unit[0]
	current_topic2=Global.unit[1]
	populate_topic(current_topic)
	populate_topic2(current_topic2)
	if 3<Global.unit.size():
		question.visible=true
	$CanvasLayer/VBoxContainer2/Question/AudioStreamPlayer.stream=Global.unit[3]
	else:
		question.visible=false
	if Global.unit.size() > 4:
		videos = Global.unit[4]
		multimedia.visible = true
	else:
		multimedia.visible = false
	$CanvasLayer/WordsButton.text = languages[current_language]["Words"]
	$CanvasLayer/SentencesButton.text = languages[current_language]["Sentences"]
	$CanvasLayer/VBoxContainer/GoBack.text = languages[current_language]["GoBack"]
	$CanvasLayer/VBoxContainer/MainMenu.text = languages[current_language]["MainMenu"]
func populate_topic(topic: Dictionary) -> void:
	for item in topic:
		var row = preload("res://scenes/item_row.tscn").instantiate()
		row.setup2(topic[item])
		$CanvasLayer/ScrollContainer/VBoxContainer.add_child(row)
func populate_topic2(topic: Dictionary) -> void:
	for item in topic:
		var row = preload("res://scenes/item_row.tscn").instantiate()
		row.setup2(topic[item])
```

<<OwlyFly V1.0>>

```
	$CanvasLayer/ScrollContainer2/VBoxContainer.add_child(row)
# Populate overlay with scaled content
func populate_overlay(topic: Dictionary):
	for child in overlay_vbox.get_children():
		child.queue_free()
	for item in topic:
		var row = preload("res://scenes/item_row.tscn").instantiate()
		row.setup2(topic[item], true)  # Pass true for overlay mode
		row.custom_minimum_size = Vector2(1500, 100)  # Adjust width to match
container
		overlay_vbox.add_child(row)
func _on_go_back_pressed() -> void:
	Global.inmenu=true
	if pressed == 0:
		pressed = 1
		$AudioSFX.play()
		await $AudioSFX.finished
		Global.inmenu=true
		#print (Global.unit[2])
		match Global.unit[2]:
			"baby": get_tree().change_scene_to_file("res://scenes/class_baby.tscn")
			"small":
get_tree().change_scene_to_file("res://scenes/class_small.tscn")
			"middle":
get_tree().change_scene_to_file("res://scenes/class_middle.tscn")
			"big": get_tree().change_scene_to_file("res://scenes/class_big.tscn")
			"demo": get_tree().change_scene_to_file("res://scenes/demobaby.tscn")
			"extra1":
get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
			"summercamp":
get_tree().change_scene_to_file("res://scenes/class_camps.tscn")
			_: get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_main_menu_pressed() -> void:
	if pressed == 0:
		pressed = 1
		$AudioSFX.play()
		await $AudioSFX.finished
		Global.inmenu=true
	get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_question_pressed() -> void:
	if $CanvasLayer/VBoxContainer2/Question/AudioStreamPlayer.stream:
		# Add visual feedback
		_create_press_effect()
		# Play audio
	$CanvasLayer/VBoxContainer2/Question/AudioStreamPlayer.play()
func _create_press_effect() -> void:
	var tween = create_tween()
	tween.tween_property($TextureButton, "modulate", Color(0.8, 0.8, 0.8, 1), 0.1)
	tween.tween_property($TextureButton, "modulate", Color.WHITE, 0.1)
func _on_words_button_pressed() -> void:
	populate_overlay(current_topic)
	overlay_panel.visible = true
func _on_close_pressed() -> void:
	overlay_panel.visible = false
func _on_sentences_button_pressed() -> void:
	populate_overlay(current_topic2)
```

```
      overlay_panel.visible = true
func _on_multimedia_pressed() -> void:
      if pressed == 0:
            pressed = 1
            $AudioSFX.play()
            await $AudioSFX.finished
            pressed=0
            # Create and show the multimedia viewer
            var multimedia_viewer =
preload("res://scenes/multimedia_viewer.tscn").instantiate()
            multimedia_viewer.set_videos(videos)
            add_child(multimedia_viewer)
```

item_row.gd:

```
extends HBoxContainer
var overlay_scale: float = 1.0
func _ready():
      # Set button size constraints
      #$TextureButton.custom_minimum_size = Vector2(100, 100)
      #$TextureButton.ignore_texture_size = true
      _update_sizes()
func _update_sizes() -> void:
      # Update sizes based on current scale
      $TextureButton.custom_minimum_size = Vector2(100, 100) * overlay_scale
      $ItemName.add_theme_font_size_override("font_size", 30 * overlay_scale)  # Adjust
base size as needed
      # Configure texture scaling
      $TextureButton.stretch_mode = TextureButton.STRETCH_SCALE
func setup(item_data: Dictionary) -> void:
      # Set item name
      $ItemName.text = item_data.get("name", "")
      # Load and display image
      var image_path = item_data.get("image", "")
      if FileAccess.file_exists(image_path):
            var image = Image.load_from_file(image_path)
            var texture = ImageTexture.create_from_image(image)
            $TextureButton.texture_normal = texture
      else:
            push_warning("Image not found: ", image_path)
      # Load audio
      var audio_path = item_data.get("audio", "")
      if FileAccess.file_exists(audio_path):
            var audio_file = FileAccess.open(audio_path, FileAccess.READ)
            if audio_file:
                  $TextureButton/AudioStreamPlayer.stream = AudioStreamMP3.new()
                  $TextureButton/AudioStreamPlayer.stream.data =
audio_file.get_buffer(audio_file.get_length())
      else:
            push_warning("Audio not found: ", audio_path)
func setup2(item_data: Dictionary, is_overlay: bool = false) -> void:
      if is_overlay:
            overlay_scale = 3.0
            _update_sizes()
      # Set item name
      $ItemName.text = item_data.get("name", "")
      # Load and display image
      $TextureButton.texture_normal = item_data["image"]
```

<<OwlyFly V1.0>>

```
	# Load audio
	#$TextureButton/AudioStreamPlayer.stream = AudioStreamMP3.new()
	$TextureButton/AudioStreamPlayer.stream = item_data["audio"]
func _on_texture_button_pressed() -> void:
	if $TextureButton/AudioStreamPlayer.stream:
		# Add visual feedback
		_create_press_effect()
		# Play audio
		$TextureButton/AudioStreamPlayer.play()
func _create_press_effect() -> void:
	var tween = create_tween()
	tween.tween_property($TextureButton, "modulate", Color(0.8, 0.8, 0.8, 1), 0.1)
	tween.tween_property($TextureButton, "modulate", Color.WHITE, 0.1)
```

setup_1v_1.gd:

```
extends Control
@onready var difficulty: Button = $VBoxContainer/Difficulty
@onready var rounds: HSlider = $VBoxContainer/Rounds
@onready var time_per_round: HSlider = $VBoxContainer/TimePerRound
@onready var go_back: Button = $VBoxContainer/GoBack
@onready var main_menu: Button = $VBoxContainer/MainMenu
@onready var image_holder: TextureRect = $HBoxContainer/ImageHolder
@onready var rounds_label: Label = $VBoxContainer/RoundsLabel
@onready var time_per_round_label: Label = $VBoxContainer/TimePerRoundLabel
var languages = {
	0: {  # 英语 English
		"MainMenu": "          Main Menu          ",
		"GoBack": " Go Back ",
		"Versus": "Versus Mode",
		"Difficulty": " Difficulty: ",
		"Easy": " Easy ",
		"Normal": " Normal ",
		"Hard": " Hard ",
		"Rounds": " Number Of Rounds : ",
		"Play": " Play ",
		"Time": " Time Per Round: "
	},
	1: {  # 中文 Chinese
		"MainMenu": "       主菜单       ",
		"GoBack": " 返回 ",
		"Versus": " 对战模式 ",
		"Difficulty": " 难度: ",
		"Easy": " ex ",
		"Normal": " 普通 ",
		"Hard": " 困难 ",
		"Rounds": " 回合数: ",
		"Play": " 开始游戏 ",
		"Time": " 每回合时间: "
	}
}
var current_language = Global.language
var images =[preload("res://images/1v1backgrounds/catwindow.jpg"),
	preload("res://images/1v1backgrounds/tree.jpeg"),
	preload("res://images/1v1backgrounds/forest2.jpeg")
	]
var pressed
```

<<OwlyFly V1.0>>

```
var difficulty_level
var totalrounds
var timeperround
var image1: Texture2D
var image2: Texture2D
var current_index=0
# Called when the node enters the scene tree for the first time.
func _ready() -> void:
	Global.inmenu=true
	$AudioSFX.volume_db = linear_to_db(Global.voice_volume / 10.0)
	pressed = 0
	difficulty_level = Global.difficulty1v1
	totalrounds=rounds.value
	timeperround=time_per_round.value
	image_holder.texture=images[current_index]
	go_back.text = languages[current_language]["GoBack"]
	main_menu.text = languages[current_language]["MainMenu"]
	if difficulty_level == 1:
		difficulty.text=
languages[current_language]["Difficulty"]+languages[current_language]["Normal"]
	else:

	difficulty.text=languages[current_language]["Difficulty"]+languages[current_languag
e]["Hard"]#
	rounds_label.text=languages[current_language]["Rounds"]+str(totalrounds)
	time_per_round_label.text=languages[current_language]["Time"]+str(timeperround)
	$"1vs1head/Label".text=languages[current_language]["Versus"]
	$PlayButton.text=languages[current_language]["Play"]
	image1=image_holder.texture
	image2=image_holder.texture
func _process(delta: float) -> void:
	pass
func _on_difficulty_pressed() -> void:
	if difficulty_level==1:
		difficulty_level=2
	elif difficulty_level==2:
		difficulty_level=0
	elif difficulty_level==0:
		difficulty_level=1
	else:
		difficulty_level=1
	if difficulty_level == 1:
		difficulty.text=
languages[current_language]["Difficulty"]+languages[current_language]["Normal"]
	elif difficulty_level==2:

	difficulty.text=languages[current_language]["Difficulty"]+languages[current_languag
e]["Hard"]
	else:

	difficulty.text=languages[current_language]["Difficulty"]+languages[current_languag
e]["Easy"]
	Global.difficulty1v1=difficulty_level
func _on_go_back_pressed() -> void:
	if pressed==0:
		pressed=1
```

<<OwlyFly V1.0>>

```
            $AudioSFX.play()
            await $AudioSFX.finished
            match Global.unit[2]:
                  "baby": get_tree().change_scene_to_file("res://scenes/class_baby.tscn")
                  "small":
get_tree().change_scene_to_file("res://scenes/class_small.tscn")
                  "middle":
get_tree().change_scene_to_file("res://scenes/class_middle.tscn")
                  "big": get_tree().change_scene_to_file("res://scenes/class_big.tscn")
                  "demo": get_tree().change_scene_to_file("res://scenes/demobaby.tscn")
                  "extra1":
get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
                  "summercamp":
get_tree().change_scene_to_file("res://scenes/class_camps.tscn")
                  _: get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_main_menu_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_rounds_value_changed(value: float) -> void:
      totalrounds=value
      rounds_label.text=languages[current_language]["Rounds"]+str(totalrounds)
func _on_time_per_round_value_changed(value: float) -> void:
      timeperround=value
      time_per_round_label.text=languages[current_language]["Time"]+str(timeperround)
func _on_play_button_pressed() -> void:
      if pressed==0:
            pressed=1
            var versus = [totalrounds, timeperround, difficulty_level, image1, image2]
            Global.versusdata = versus
            $AudioSFX.play()
            await $AudioSFX.finished
            get_tree().change_scene_to_file("res://scenes/1_vs_1.tscn")
func _on_arrow_left_pressed():
      current_index = wrapi(current_index - 1, 0, images.size())
      image_holder.texture=images[current_index]
      image1=image_holder.texture
      image2=image_holder.texture
func _on_arrow_right_pressed():
      current_index = wrapi(current_index + 1, 0, images.size())
      image_holder.texture=images[current_index]
      image1=image_holder.texture
      image2=image_holder.texture
```

1_vs_1.gd:

```
extends Node2D
# Preload the square scene
const Square = preload("res://scenes/square.tscn")
var languages = {
      0: {  # 英语 English
            "Round": "Round ",
            "Go": "GO!",
            "MainMenu": " Main Menu ",
      },
      1: {  # 中文 Chinese
```

```
            "Round": "回合 ",
            "Go": "开始!",
            "MainMenu": " 主菜单 ",
      }
}
var current_language = Global.language
# Game settings
var current_round := 0
var max_rounds := 3
var round_time := 10.0
var correct_topic: String
var screen_size: Vector2
# Scores: [left, right]
var round_points := [0, 0]    # Points earned in current round
var round_wins := [0, 0]      # Rounds won by each player
var game_active := false
var current_topic_dict: Dictionary
@onready var timer = $RoundTimer
@onready var audio_player = $AudioStreamPlayer
@onready var topic_label = $UILayer/TopicLabel
@onready var timer_label = $UILayer/TimerLabel
@onready var left_score_label = $UILayer/HBoxContainer/LeftScore
@onready var right_score_label = $UILayer/HBoxContainer/RightScore
@onready var round_label = $UILayer/RoundLabel
@onready var wins_label = $UILayer/WinnerLabel
@onready var wins_label_2: Label = $UILayer/WinnerLabel2
@onready var correct_sound = $CorrectSound
@onready var wrong_sound = $WrongSound
@onready var go_label = $UILayer/GoLabel
# For visual feedback
var left_flash: ColorRect
var right_flash: ColorRect
var active_squares: Array = []
var difficulty = 1
var difficulty_settings = {
      0: {"spawn_delay": 0.6, "spawn_count": 3, "speed_multiplier": 0.4},
      1: {"spawn_delay": 0.4, "spawn_count": 4, "speed_multiplier": 0.6},
      2: {"spawn_delay": 0.3, "spawn_count": 5, "speed_multiplier": 1.0}
}
# Ad system variables
var _interstitial_ad : InterstitialAd
var _ad_view : AdView
var pressed = 0  # For tracking button presses
var pressed2 = 0
func _ready():
      Global.showad = false
      screen_size = get_viewport_rect().size
      print("=== GAME STARTED ===")
      Global.inmenu = false
      var sound = preload("res://audio/effects/funnyrun.mp3")
      $music.stream=sound
      $music.volume_db=linear_to_db(Global.music_volume / 10.0)
      $AudioStreamPlayer.volume_db=linear_to_db(Global.voice_volume / 10.0)
      $WrongSound.volume_db=linear_to_db(Global.voice_volume / 10.0)
      $CorrectSound.volume_db=linear_to_db(Global.voice_volume / 10.0)
      current_language = Global.language
```

<<OwlyFly V1.0>>

```
	$UILayer/MainMenu.text = languages[current_language]["MainMenu"]
	# Initialize ads if needed
	if Global.is_premium_user == false && (OS.get_name() == "Android"):
		MobileAds.initialize()
		var request_configuration := RequestConfiguration.new()
		request_configuration.tag_for_child_directed_treatment =
RequestConfiguration.TagForChildDirectedTreatment.TRUE
		MobileAds.set_request_configuration(request_configuration)
		_create_ad_view()
		_on_load_intersicial()
	# Hide GO! label initially
	go_label.visible = false
	go_label.text = languages[current_language]["Go"]
	# Load topic resource
	if Global.versusdata == []:
		var topic_resource = load("res://topics/smallunit1dif1.tres")
		current_topic_dict = _process_topic_resource(topic_resource)
	else:
		current_topic_dict = Global.custom_topics[Global.current_topic]
		max_rounds = Global.versusdata[0]
		round_time = Global.versusdata[1]
		difficulty = Global.difficulty1v1
	# Share topic dictionary
	SquareScript.current_topic_dict = current_topic_dict
	# Create player areas
	create_player_areas()
	update_score_labels()
	# Clear topic label initially
	topic_label.text = ""
	# Start first round after delay
	await get_tree().create_timer(1.0).timeout
	start_new_round()
func _process(delta):
	if timer.time_left > 0 and game_active:
		timer_label.text = str(ceil(timer.time_left))
	else:
		timer_label.text = ""
	if Global.showad:
		# Handle ad showing logic
		if Global.is_premium_user == false:
			if Global.region == "NoChina" || Global.region == "China":
				destroy_ad_view()
				if _interstitial_ad:
					_interstitial_ad.show()
			Global.showad = false
		Global.victory = false
		# Change scene based on current unit
		if Global.unit[2] == "main":
			get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
		else:
			match Global.unit[2]:
				"baby":
get_tree().change_scene_to_file("res://scenes/class_baby.tscn")
				"small":
get_tree().change_scene_to_file("res://scenes/class_small.tscn")
```

<<OwlyFly V1.0>>

```
                        "middle":
get_tree().change_scene_to_file("res://scenes/class_middle.tscn")
                        "big":
get_tree().change_scene_to_file("res://scenes/class_big.tscn")
                        "demo":
get_tree().change_scene_to_file("res://scenes/demobaby.tscn")
                        "extra1":
get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
                        "summercamp":
get_tree().change_scene_to_file("res://scenes/class_camps.tscn")
                        _: get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func create_player_areas():
      # Left player area
      var left_area = Area2D.new()
      left_area.position = Vector2(0, 0)
      left_area.name = "LeftArea"
      var left_collision = CollisionShape2D.new()
      left_collision.shape = RectangleShape2D.new()
      left_collision.shape.size = Vector2(screen_size.x / 2, screen_size.y)
      left_collision.position = Vector2(screen_size.x / 4, screen_size.y / 2)
      add_child(left_area)
      left_area.add_child(left_collision)
      # Add background to left area
      var left_bg = Sprite2D.new()
      if Global.versusdata == []:
            left_bg.texture = preload("res://images/city.png")
      else:
            left_bg.texture = Global.versusdata[3]
      left_bg.centered = false
      left_bg.position = Vector2(0, 0)
      left_bg.scale = Vector2(
            (screen_size.x / 2) / left_bg.texture.get_width(),
            screen_size.y / left_bg.texture.get_height()
      )
      left_bg.z_index = -1
      left_area.add_child(left_bg)
      # Create and add flash effect for left area
      self.left_flash = ColorRect.new()
      left_flash.color = Color(1, 0, 0, 0.5)
      left_flash.size = Vector2(screen_size.x / 2, screen_size.y)
      left_flash.visible = false
      left_flash.z_index = 10
      left_area.add_child(left_flash)
      # Right player area
      var right_area = Area2D.new()
      right_area.position = Vector2(screen_size.x / 2, 0)
      right_area.name = "RightArea"
      var right_collision = CollisionShape2D.new()
      right_collision.shape = RectangleShape2D.new()
      right_collision.shape.size = Vector2(screen_size.x / 2, screen_size.y)
      right_collision.position = Vector2(screen_size.x / 4, screen_size.y / 2)
      add_child(right_area)
      right_area.add_child(right_collision)
      # Add background to right area
      var right_bg = Sprite2D.new()
      if Global.versusdata == []:
```

```
            right_bg.texture = preload("res://images/citytunnel2.png")
      else:
            right_bg.texture = Global.versusdata[4]
      right_bg.centered = false
      right_bg.position = Vector2(0, 0)
      right_bg.scale = Vector2(
            (screen_size.x / 2) / right_bg.texture.get_width(),
            screen_size.y / right_bg.texture.get_height()
      )
      right_bg.z_index = -1
      right_area.add_child(right_bg)
      # Create and add flash effect for right area
      self.right_flash = ColorRect.new()
      right_flash.color = Color(1, 0, 0, 0.5)
      right_flash.size = Vector2(screen_size.x / 2, screen_size.y)
      right_flash.visible = false
      right_flash.z_index = 10
      right_area.add_child(right_flash)
func update_score_labels():
      left_score_label.text = str(round_points[0])
      right_score_label.text = str(round_points[1])
      wins_label.text = str(round_wins[0])
      wins_label_2.text = str(round_wins[1])
func _process_topic_resource(topic_data: Resource) -> Dictionary:
      var result = {}
      for entry in topic_data.entries:
            result[entry.name] = {
                  "name": entry.name,
                  "image": load(entry.image_path),
                  "audio": load(entry.audio_path)
            }
      return result
func start_new_round():
      round_points = [0, 0]
      update_score_labels()
      game_active = true
      current_round += 1
      round_label.text = languages[current_language]["Round"] + "%d/%d" % [current_round,
max_rounds]
      var topics = current_topic_dict.keys()
      correct_topic = topics[randi() % topics.size()]
      topic_label.text = correct_topic
      await play_topic_audio()
      show_go_animation()
      timer.start(round_time)
      $music.play()
      spawn_squares()
func play_topic_audio():
      var stream = current_topic_dict[correct_topic]["audio"]
      for i in 3:
            audio_player.stream = stream
            audio_player.play()
            await audio_player.finished
            await get_tree().create_timer(0.5).timeout
func show_go_animation():
      go_label.visible = true
```

```
	go_label.modulate.a = 1.0
	var tween = create_tween()
	tween.tween_property(go_label, "modulate:a", 0.0, 1.0).set_ease(Tween.EASE_OUT)
	tween.tween_callback(func(): go_label.visible = false)
	await tween.finished
func spawn_squares():
	var settings = difficulty_settings[difficulty]
	var spawn_delay = settings["spawn_delay"]
	var spawn_count = settings["spawn_count"]
	while timer.time_left > 0 and game_active:
		for i in spawn_count:
			create_square(generate_spawn_point(0), generate_direction(0), 0)
			create_square(generate_spawn_point(1), generate_direction(1), 1)
		await get_tree().create_timer(spawn_delay).timeout
func generate_spawn_point(side: int) -> Vector2:
	var margin = 50
	var spawn_type = randi() % 5
	match spawn_type:
		0: # Top edge
			if side == 0:
				return Vector2(randf_range(margin, screen_size.x/2 - margin), -
margin)
			else:
				return Vector2(randf_range(screen_size.x/2 + margin,
screen_size.x - margin), -margin)
		1: # Bottom edge
			if side == 0:
				return Vector2(randf_range(margin, screen_size.x/2 - margin),
screen_size.y + margin)
			else:
				return Vector2(randf_range(screen_size.x/2 + margin,
screen_size.x - margin), screen_size.y + margin)
		2: # Left edge
			if side == 0:
				return Vector2(-margin, randf_range(margin, screen_size.y -
margin))
			else:
				return Vector2(randf_range(screen_size.x/2 + margin,
screen_size.x - margin), -margin)
		3: # Right edge
			if side == 1:
				return Vector2(screen_size.x + margin, randf_range(margin,
screen_size.y - margin))
			else:
				return Vector2(randf_range(margin, screen_size.x/2 - margin),
screen_size.y + margin)
		4: # Center area
			if side == 0:
				return Vector2(
					randf_range(margin, screen_size.x/2 - margin),
					randf_range(screen_size.y * 0.3, screen_size.y * 0.7)
				)
			else:
				return Vector2(
					randf_range(screen_size.x/2 + margin, screen_size.x -
margin),
```

```
                              randf_range(screen_size.y * 0.3, screen_size.y * 0.7)
                        )
      if side == 0:
            return Vector2(randf_range(margin, screen_size.x/2 - margin), -margin)
      else:
            return Vector2(randf_range(screen_size.x/2 + margin, screen_size.x - margin),
-margin)
func generate_direction(side: int) -> Vector2:
      var direction_type = randi() % 3
      var center_x = screen_size.x / 4 if side == 0 else screen_size.x * 0.75
      var center = Vector2(center_x, screen_size.y / 2)
      match direction_type:
            0: # Toward center with variation
                  var variation_x = randf_range(-screen_size.x * 0.3, screen_size.x *
0.3)
                  var variation_y = randf_range(-screen_size.y * 0.3, screen_size.y *
0.3)
                  var target = center + Vector2(variation_x, variation_y)
                  return (target - center).normalized()
            1: # Away from center
                  var angle = randf_range(0, TAU)
                  return Vector2(cos(angle), sin(angle))
            2: # Random direction
                  return Vector2(randf_range(-1, 1), randf_range(-1, 1)).normalized()
      return Vector2.ZERO
func create_square(pos: Vector2, dir: Vector2, side: int):
      var settings = difficulty_settings[difficulty]
      var square = Square.instantiate()
      square.position = pos
      square.direction = dir
      square.speed = randf_range(150, 400) * settings["speed_multiplier"]
      square.player_side = side
      # Get all topics and create a weighted list where correct_topic has double weight
      var topics = current_topic_dict.keys()
      var weighted_topics = []
      # Add each topic once, and the correct topic an additional time
      for topic in topics:
            weighted_topics.append(topic)
            if topic == correct_topic:
                  weighted_topics.append(topic)  # Extra entry for correct topic
      # Select a random topic from the weighted list
      square.topic_name = weighted_topics[randi() % weighted_topics.size()]
      add_child(square)
      active_squares.append(square)
func handle_square_click(square: SquareScript):
      if square.topic_name == correct_topic:
            correct_sound.play()
            round_points[square.player_side] += 1
            update_score_labels()
            destroy_square(square)
      else:
            wrong_sound.play()
            show_flash(square.player_side)
func destroy_square(square: SquareScript):
      if is_instance_valid(square) and square in active_squares:
            square.break_square()
```

<<OwlyFly V1.0>>

```
            active_squares.erase(square)
func show_flash(side: int):
      if side == 0:
            left_flash.visible = true
            await get_tree().create_timer(0.5).timeout
            left_flash.visible = false
      else:
            right_flash.visible = true
            await get_tree().create_timer(0.5).timeout
            right_flash.visible = false
func _on_round_timer_timeout():
      print("FINISH THE ROUND")
      game_active = false
      $music.stop()
      topic_label.text = ""
      await get_tree().create_timer(1.0).timeout
      if round_points[0] > round_points[1]:
            round_wins[0] += 1
      elif round_points[1] > round_points[0]:
            round_wins[1] += 1
      else:
            round_wins[0] += 1
            round_wins[1] += 1
      update_score_labels()
      for child in get_children():
            if child is SquareScript:
                  child.break_square()
      await get_tree().create_timer(1.5).timeout
      if current_round < max_rounds:
            show_go_animation()
            await get_tree().create_timer(1.0).timeout
            start_new_round()
      else:
            end_game()
func end_game():
      var winner_side = -1
      if round_wins[0] > round_wins[1]:
            winner_side = 0
      elif round_wins[1] > round_wins[0]:
            winner_side = 1
      else:
            winner_side = 2
      topic_label.text = "Game Over!"
      timer_label.text = "Final Score\nLeft: %d - Right: %d" % [round_wins[0],
round_wins[1]]
      game_active = false
      show_victory_screen(winner_side)
func show_victory_screen(winner_side: int):
      var victory_scene = preload("res://scenes/victory1v1.tscn")
      var victory_instance = victory_scene.instantiate()
      var screen_size = get_viewport().get_visible_rect().size
      var container = victory_instance.get_node("Container")
      var finishsound = preload("res://audio/effects/winning-218995.mp3")
      $music.stream=finishsound
      $music.play()
      if container:
```

<<OwlyFly V1.0>>

```
            if winner_side == 2:
                  victory_instance.isdraw = true
                  container.position = Vector2.ZERO
                  container.size = screen_size
                  var box = victory_instance.get_node("ButtonsContainer")
                  box.visible = false
            else:
                  container.position = Vector2(0, 0) if winner_side == 0 else
Vector2(screen_size.x / 2, 0)
                  container.scale.x = 0.5
                  var box =
victory_instance.get_node("Container/Anchor/ButtonsContainer")
                  box.visible = false
      var level_label = victory_instance.get_node("Container/Anchor/Shield/Level")
      if level_label:
            if winner_side == 0:
                  level_label.text = "LEFT PLAYER WINS!"
            elif winner_side == 1:
                  level_label.text = "RIGHT PLAYER WINS!"
            else:
                  level_label.text = "DRAW!"
      add_child(victory_instance)
# ========== Ad System Functions ==========
func _create_ad_view() -> void:
      if _ad_view:
            destroy_ad_view()
      var unit_id : String
      if OS.get_name() == "Android":
            unit_id = "ca-app-pub-3940256099942544~3347511713"
      elif OS.get_name() == "iOS":
            unit_id = "ca-app-pub-3940256099942544/6300978111"
      _ad_view = AdView.new(unit_id, AdSize.BANNER, AdPosition.Values.TOP)
      #var ad_request := AdRequest.new() #CRITICAL here we deactivate banners
      #_ad_view.load_ad(ad_request)
func destroy_ad_view() -> void:
      if _ad_view:
            _ad_view.destroy()
            _ad_view = null
func _on_load_intersicial():
      if _interstitial_ad:
            _interstitial_ad.destroy()
            _interstitial_ad = null
      var unit_id : String
      if OS.get_name() == "Android":
            unit_id = "ca-app-pub-3940256099942544~3347511713"
      elif OS.get_name() == "iOS":
            unit_id = "ca-app-pub-3940256099942544/6300978111"
      var interstitial_ad_load_callback := InterstitialAdLoadCallback.new()
      interstitial_ad_load_callback.on_ad_failed_to_load = func(adError : LoadAdError) ->
void:
            print(adError.message)
      interstitial_ad_load_callback.on_ad_loaded = func(interstitial_ad : InterstitialAd)
-> void:
            print("interstitial ad loaded" + str(interstitial_ad._uid))
            _interstitial_ad = interstitial_ad
```

<<OwlyFly V1.0>>

```
      InterstitialAdLoader.new().load(unit_id, AdRequest.new(),
interstitial_ad_load_callback)
# ========== End of Ad System Functions ==========
func _on_main_menu_pressed() -> void:
      if pressed2 == 0:
            pressed2 = 1
            Global.good_obstacles_collected = 0
            if Global.is_premium_user == false:
                  if Global.region == "NoChina" or Global.region=="China":
                        # Existing AdMob cleanup
                        destroy_ad_view()
                        if _interstitial_ad:
                              _interstitial_ad.show()
            Global.showad = false
            Global.victory = false
            get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
```

victory_1v_1.gd:

```
extends CanvasLayer
@onready var background_particles = $Container/BackgroundParticles
@onready var wing_particles = $Container/Anchor/Shield/WingR/WingParticles
@onready var wing_particles_2 = $Container/Anchor/Shield/WingL/WingParticles2
@onready var container = $Container
@onready var rays = $Container/Anchor/Rays
@onready var shield = $Container/Anchor/Shield
@onready var level = $Container/Anchor/Shield/Level
@onready var wing_r = $Container/Anchor/Shield/WingR
@onready var wing_l = $Container/Anchor/Shield/WingL
@onready var buttons_container = $ButtonsContainer
@onready var buttons_container2: VBoxContainer = $Container/Anchor/ButtonsContainer
var languages = {
      0: {  # English
            "PlayAgain": " Play Again ",
            "MainMenu": " Main Menu ",
            "GoBack": "Go Back",
            "Victory": " VICTORY! ",
            "Draw": " DRAW! "
      },
      1: {  # Chinese
            "PlayAgain": " 再玩一次 ",
            "MainMenu": " 主菜单 ",
            "GoBack": "返回",
            "Victory": " 胜利! ",
            "Draw": " 平局! "
      }
}
var screen_size: Vector2
var current_language # Default to English
var isdraw=false
var tween: Tween
var tween2: Tween
var tween_ray: Tween
var tween_buttons: Tween
var tween_buttons2: Tween
func _ready() -> void:
      get_viewport().connect("size_changed", Callable(self, "_on_viewport_size_changed"))
      screen_size = get_viewport().get_visible_rect().size
```

<<OwlyFly V1.0>>

```
	adjust_layout()
	current_language = Global.language
	$ButtonsContainer/Retry.text = languages[current_language]["PlayAgain"]
	$ButtonsContainer/GoBack.text = languages[current_language]["GoBack"]
	if isdraw==false:
			$Container/Anchor/Shield/Level.text =
languages[current_language]["Victory"]
	else:
		$Container/Anchor/Shield/Level.text = languages[current_language]["Draw"]
	$ButtonsContainer/MainMenu.text = languages[current_language]["MainMenu"]
	$Container/Anchor/ButtonsContainer/Retry.text =
languages[current_language]["PlayAgain"]
	$Container/Anchor/ButtonsContainer/MainMenu.text =
languages[current_language]["MainMenu"]
	$Container/Anchor/ButtonsContainer/GoBack.text =
languages[current_language]["GoBack"]
	background_particles.emitting = false
	wing_particles.emitting = false
	wing_particles_2.emitting = false
	rays.scale = Vector2.ZERO
	shield.scale = Vector2.ZERO
	level.self_modulate.a = 0.0
	wing_r.scale = Vector2.ZERO
	wing_l.scale = Vector2.ZERO
	for child in buttons_container.get_children():
		child.modulate.a = 0.0
	animate()
func _on_viewport_size_changed():
	adjust_layout()
func adjust_layout():
	var new_size = get_viewport().get_visible_rect().size
	# Update container size if it exists
	if has_node("Container"):
		var container = $Container
		var current_width = container.size.x
		# Check if we're in half-screen mode (current width ≈ screen width / 2)
		if abs(current_width - screen_size.x / 2) < 10:
			# Maintain half-screen size
			container.size = Vector2(new_size.x / 2, new_size.y)
			# Position on correct side
			if container.position.x > 0:  # Right side
				container.position = Vector2(new_size.x / 2, 0)
			# Reposition content
			var anchor = $Container/Anchor
			if anchor:
				anchor.position = Vector2(new_size.x / 4, new_size.y / 2)
			# Reposition buttons
			var buttons = $ButtonsContainer
			if buttons:
				buttons.position = Vector2(
					new_size.x / 4 - buttons.size.x / 2,
					new_size.y - 100
				)
		else:
			# Full screen mode
			container.size = new_size
```

<<OwlyFly V1.0>>

```
                  var anchor = $Container/Anchor
                  if anchor:
                        anchor.position = new_size / 2
      # Update screen size reference
      screen_size = new_size
func animate() -> void:
      tween = create_tween().set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_CUBIC)
      tween_ray =
create_tween().set_trans(Tween.TRANS_LINEAR).set_ease(Tween.EASE_IN_OUT)
      tween2 = create_tween().set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_CUBIC)
      tween_ray.set_loops()
      tween_ray.tween_property(rays, "rotation_degrees", 360.0, 8.0).from(0.0)
      tween.tween_interval(0.4)
      # Rays & shield
      tween.parallel().tween_callback(background_particles.restart)
      tween.parallel().tween_property(rays, "scale", Vector2.ONE,
0.45).from(Vector2.ZERO)
      tween.parallel().tween_property(shield, "scale", Vector2.ONE,
1.4).set_trans(Tween.TRANS_ELASTIC).set_ease(Tween.EASE_OUT)
      tween.parallel().tween_property(shield.material, "shader_parameter/y_rot", 180.0,
1.2).set_trans(Tween.TRANS_ELASTIC).set_ease(Tween.EASE_OUT)
      # Labels
      tween.parallel().tween_property(level, "self_modulate:a", 1.0, 1.5)
      # Wings
      tween2.tween_interval(1.2)
      tween2.tween_property(wing_r, "position:x", wing_r.position.x,
0.15).from(wing_r.position.x - 150)
      tween2.parallel().tween_property(wing_l, "position:x", wing_l.position.x,
0.15).from(wing_l.position.x + 150)
      tween2.parallel().tween_property(wing_r, "scale", Vector2.ONE,
0.9).set_trans(Tween.TRANS_ELASTIC).set_ease(Tween.EASE_OUT)
      tween2.parallel().tween_property(wing_l, "scale", Vector2.ONE,
0.9).set_trans(Tween.TRANS_ELASTIC).set_ease(Tween.EASE_OUT)
      tween2.tween_callback(wing_particles.restart)
      tween2.tween_callback(wing_particles_2.restart)
      # Ribbon
      for child in buttons_container.get_children():
            tween.tween_property(child, "modulate:a", 1.0, 0.15)
            tween.tween_interval(0.05)
      tween.tween_callback(start_buttons_loop)
func start_buttons_loop() -> void:
      tween_buttons =
create_tween().set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_CUBIC)
      tween_buttons.set_loops()
      for child in buttons_container.get_children():
            tween_buttons.tween_property(child, "scale", Vector2(1.1, 1.1), 0.2)
            tween_buttons.tween_property(child, "scale", Vector2.ONE, 0.2)
      tween_buttons.tween_interval(0.4)
      tween_buttons2 =
create_tween().set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_CUBIC)
      tween_buttons2.set_loops()
      for child in buttons_container2.get_children():
            tween_buttons2.tween_property(child, "scale", Vector2(1.1, 1.1), 0.2)
            tween_buttons2.tween_property(child, "scale", Vector2.ONE, 0.2)
      tween_buttons2.tween_interval(0.4)
func _on_go_back_pressed():
```

<<OwlyFly V1.0>>

```
      print("push go back")
      print(Global.unit)
      Global.showad=true
func _on_retry_pressed() -> void:
      get_tree().change_scene_to_file("res://scenes/1_vs_1.tscn")
func _on_main_menu_pressed() -> void:
      print("push main menu")
      Global.showad=true
      Global.unit[2]="main"
```

whackamole_setup.gd:

```
extends Control
@onready var difficulty: Button = $VBoxContainer/Difficulty
@onready var rounds: HSlider = $VBoxContainer/Rounds
@onready var time_per_round: HSlider = $VBoxContainer/TimePerRound
@onready var go_back: Button = $VBoxContainer/GoBack
@onready var main_menu: Button = $VBoxContainer/MainMenu
@onready var image_holder: TextureRect = $HBoxContainer/ImageHolder
@onready var rounds_label: Label = $VBoxContainer/RoundsLabel
var languages = {
      0: {  # 英语 English
            "MainMenu": "          Main Menu          ",
            "GoBack": " Go Back ",
            "Title": "Whack-A-Mole!",
            "Difficulty": " Difficulty: ",
            "Easy": " Easy ",
            "Normal": " Normal ",
            "Hard": " Hard ",
            "Rounds": " Number Of Rounds : ",
            "Play": " Play ",
            "Time": " Time Per Round: "
      },
      1: {  # 中文 Chinese
            "MainMenu": "       主菜单       ",
            "GoBack": " 返回 ",
            "Title": " Whack-A-Mole! ",
            "Difficulty": " 难度: ",
            "Easy": "  ",
            "Normal": " 普通 ",
            "Hard": " 困难 ",
            "Rounds": " 回合数: ",
            "Play": " 开始游戏 ",
            "Time": " 每回合时间: "
      }
}
var current_language = Global.language
var images =[preload("res://images/1v1backgrounds/catwindow.jpg"),
      preload("res://images/1v1backgrounds/tree.jpeg"),
      preload("res://images/1v1backgrounds/forest2.jpeg")
      ]
var pressed
var difficulty_level
var totalrounds
var image1: Texture2D
var current_index=0
# Called when the node enters the scene tree for the first time.
```

<<OwlyFly V1.0>>

```
func _ready() -> void:
	Global.inmenu=true
	pressed = 0
	difficulty_level = Global.difficulty1v1
	totalrounds=rounds.value
	image_holder.texture=images[current_index]
	go_back.text = languages[current_language]["GoBack"]
	main_menu.text = languages[current_language]["MainMenu"]
	if difficulty_level == 1:
		difficulty.text=
languages[current_language]["Difficulty"]+languages[current_language]["Normal"]
	else:

	difficulty.text=languages[current_language]["Difficulty"]+languages[current_languag
e]["Hard"]#
	rounds_label.text=languages[current_language]["Rounds"]+str(totalrounds)
	$"1vs1head/Label".text=languages[current_language]["Title"]
	$PlayButton.text=languages[current_language]["Play"]
	image1=image_holder.texture
func _process(delta: float) -> void:
	pass
func _on_difficulty_pressed() -> void:
	if difficulty_level==1:
		difficulty_level=2
	else:
		difficulty_level=1
	if difficulty_level == 1:
		difficulty.text=
languages[current_language]["Difficulty"]+languages[current_language]["Normal"]
	else:

	difficulty.text=languages[current_language]["Difficulty"]+languages[current_languag
e]["Hard"]
	Global.difficulty1v1=difficulty_level
func _on_go_back_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
		await $AudioSFX.finished
		match Global.unit[2]:
			"baby": get_tree().change_scene_to_file("res://scenes/class_baby.tscn")
			"small":
get_tree().change_scene_to_file("res://scenes/class_small.tscn")
			"middle":
get_tree().change_scene_to_file("res://scenes/class_middle.tscn")
			"big": get_tree().change_scene_to_file("res://scenes/class_big.tscn")
			"demo": get_tree().change_scene_to_file("res://scenes/demobaby.tscn")
			"extra1":
get_tree().change_scene_to_file("res://scenes/extralevels.tscn")
			"summercamp":
get_tree().change_scene_to_file("res://scenes/class_camps.tscn")
			_: get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_main_menu_pressed() -> void:
	if pressed==0:
		pressed=1
		$AudioSFX.play()
```

<<OwlyFly V1.0>>

```
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_rounds_value_changed(value: float) -> void:
      totalrounds=value
      rounds_label.text=languages[current_language]["Rounds"]+str(totalrounds)
func _on_play_button_pressed() -> void:
      if pressed==0:
            pressed=1
            var versus = [totalrounds, 0, difficulty_level, image1]
            Global.versusdata = versus
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/whackamole.tscn")
func _on_arrow_left_pressed():
      current_index = wrapi(current_index - 1, 0, images.size())
      image_holder.texture=images[current_index]
      image1=image_holder.texture
func _on_arrow_right_pressed():
      current_index = wrapi(current_index + 1, 0, images.size())
      image_holder.texture=images[current_index]
      image1=image_holder.texture
```

whackamole.gd:

```
extends Node2D
@onready var audio_player: AudioStreamPlayer =$AudioStreamPlayer
@onready var target_label: Label = $TargetLabel
@onready var go_label: Label = $GoLabel
var languages = {
      0: {  # English
            "Obstacles": " Obstacles: ",
            "MainMenu": " Main Menu "
      },
      1: {  # Chinese
            "Obstacles": " 障碍物: ",
            "MainMenu": " 主菜单 "
      }
}
var current_language
# Game configuration
var rounds: int = 10
var difficulty: int = 1  # 0 = 3 buttons, 1 = 6 buttons
var current_topic = Global.topic  # Load from Global
var current_round: int = 0
var buttons = []
var is_transitioning = false
var pulse_tweens = []
var current_good_item: String = ""
func _ready():
      var finishsound = preload("res://audio/effects/balloon_pop.mp3")
      $SFX.stream=finishsound
      target_label.visible = false
      go_label.visible = false
      current_language = Global.language
      $MainMenu.text = languages[current_language]["MainMenu"]
      $music.volume_db=Global.music_volume
      $SFX.volume_db=Global.voice_volume
      $AudioStreamPlayer.volume_db=Global.voice_volume
```

<<OwlyFly V1.0>>

```
	$music.play()
	Global.inmenu = false
	current_topic = Global.custom_topics[Global.current_topic]
	rounds = Global.versusdata[0]
	if Global.difficulty1v1 ==1:
		difficulty=0
	else:
		difficulty=1
	$TextureRect3.texture=Global.versusdata[3]
	start_round()
func start_round():
	is_transitioning = true
	target_label.visible = false
	go_label.visible = false
	# Clear existing buttons and tweens
	await clear_buttons()
	# Update round counter
	current_round += 1
	if current_round > rounds:
		show_victory()
		return
	# Select random good item
	var topic_keys = current_topic.keys()
	current_good_item = topic_keys[randi() % topic_keys.size()]
	# Show target label immediately and keep it visible
	target_label.text = current_good_item
	target_label.visible = true
	target_label.modulate = Color(1, 1, 1, 1)
	target_label.scale = Vector2(1, 1)
	# Determine number of buttons based on difficulty
	var num_buttons = 3 if difficulty == 0 else 6
	# Generate button keys - always include good item
	var button_keys = [current_good_item]
	# Fill remaining slots with random items (can include duplicates)
	for i in range(num_buttons - 1):
		# Get random item that's not the current good item
		var random_item
		var attempts = 0
		while attempts < 10:  # Prevent infinite loops
			random_item = topic_keys[randi() % topic_keys.size()]
			# Allow duplicates but not the good item
			if random_item != current_good_item:
				break
			attempts += 1
		button_keys.append(random_item)
	# Shuffle positions
	button_keys.shuffle()
	# Get positions based on difficulty
	var positions = calculate_positions()
	# Create buttons
	for i in range(num_buttons):
		var button_scene = preload("res://scenes/color_button.tscn")
		var button = button_scene.instantiate()
		button.setup(current_topic[button_keys[i]])
		button.position = positions[i]
		button.scale = Vector2.ZERO
```

```
            button.item_selected.connect(_on_item_selected)
            button.disabled = true
            add_child(button)
            buttons.append(button)
      # Animate buttons popping in
      await animate_buttons_in()
      # Show GO! label with animation
      go_label.visible = true
      go_label.modulate = Color(1, 1, 1, 0)
      go_label.scale = Vector2(2, 2)
      var go_tween = create_tween()
      go_tween.tween_property(go_label, "modulate", Color(1, 1, 1, 1), 0.3)
      go_tween.parallel().tween_property(go_label, "scale", Vector2(1, 1), 0.3)
      go_tween.chain().tween_property(go_label, "scale", Vector2(1.2, 1.2), 0.2)
      go_tween.chain().tween_property(go_label, "modulate", Color(1, 1, 1, 0), 0.3)
      go_tween.parallel().tween_property(go_label, "scale", Vector2(0.5, 0.5), 0.3)
      # Enable buttons immediately after they appear
      for button in buttons:
            button.disabled = false
      # FIX: Play audio immediately without delay
      play_audio(current_good_item)
      # Add idle pulse animation to all buttons
      start_idle_pulse()
      is_transitioning = false
# NEW: Simplified and reliable audio playback function
func play_audio(item_name):
      if current_topic.has(item_name):
            # Stop any current playback
            audio_player.stop()
            # Set and play the audio
            audio_player.stream = current_topic[item_name]["audio"]
            audio_player.play()
            # Play 2 more times with delays
            await get_tree().create_timer(audio_player.stream.get_length() + 0.5).timeout
            audio_player.play()
            await get_tree().create_timer(audio_player.stream.get_length() + 0.5).timeout
            audio_player.play()
func animate_buttons_in():
      var tween = create_tween().set_parallel(true)
      for button in buttons:
            tween.tween_property(button, "scale", Vector2(1, 1), 0.5)\
      .set_ease(Tween.EASE_OUT).set_trans(Tween.TRANS_BACK)
            tween.tween_property(button, "rotation", randf_range(-0.1, 0.1), 0.5)
      await tween.finished
func start_idle_pulse():
      # Clear any existing pulse tweens
      for tween in pulse_tweens:
            if tween.is_valid():
                  tween.kill()
      pulse_tweens.clear()
      # Create new pulse tweens
      for button in buttons:
            var pulse_tween = create_tween().set_loops()
            pulse_tween.tween_property(button, "scale", Vector2(1.15, 1.15), 0.6)\
                  .set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_SINE)
            pulse_tween.tween_property(button, "scale", Vector2(1.0, 1.0), 0.6)\
```

<<OwlyFly V1.0>>

```
      .set_ease(Tween.EASE_IN_OUT).set_trans(Tween.TRANS_SINE)
            pulse_tweens.append(pulse_tween)
func clear_buttons():
      # Stop all pulse animations
      for tween in pulse_tweens:
            if tween.is_valid():
                  tween.kill()
      pulse_tweens.clear()
      # Stop any playing audio
      audio_player.stop()
      # Hide target label when clearing buttons
      target_label.visible = false
      # Animate buttons popping out if they exist
      if buttons.size() > 0:
            var tween = create_tween().set_parallel(true)
            for button in buttons:
                  tween.tween_property(button, "scale", Vector2(0, 0), 0.4)\
                        .set_ease(Tween.EASE_IN).set_trans(Tween.TRANS_BACK)
                  tween.tween_property(button, "rotation", 0, 0.4)
                  tween.tween_property(button, "modulate", Color(1, 1, 1, 0), 0.4)
            await tween.finished
            # Remove buttons after animation
            for button in buttons:
                  button.queue_free()
            buttons.clear()
      # Add delay between rounds
      await get_tree().create_timer(0.4).timeout
func calculate_positions():
      var viewport = get_viewport_rect().size
      var positions = []
      if difficulty == 0:  # 3 buttons
            positions = [
                  Vector2(viewport.x * 0.25 - 150, viewport.y / 2 - 150),  # Left
                  Vector2(viewport.x * 0.5 - 150, viewport.y / 2 - 150),   # Center
                  Vector2(viewport.x * 0.75 - 150, viewport.y / 2 - 150)  # Right
            ]
      else:  # 6 buttons
            positions = [
                  # Top row
                  Vector2(viewport.x * 0.2 - 150, viewport.y * 0.3 - 150),
                  Vector2(viewport.x * 0.5 - 150, viewport.y * 0.3 - 150),
                  Vector2(viewport.x * 0.8 - 150, viewport.y * 0.3 - 150),
                  # Bottom row
                  Vector2(viewport.x * 0.2 - 150, viewport.y * 0.7 - 150),
                  Vector2(viewport.x * 0.5 - 150, viewport.y * 0.7 - 150),
                  Vector2(viewport.x * 0.8 - 150, viewport.y * 0.7 - 150)
            ]
      return positions
func _on_item_selected(button):
      if is_transitioning:
            return
      $SFX.play()
      is_transitioning = true
      var item_name = button.item_data["name"]
      if item_name == current_good_item:
            # Apply effects only to the pressed button
```

<<OwlyFly V1.0>>

```
            for tween in pulse_tweens:
                  if tween.is_valid():
                        tween.kill()
            var success_tween = create_tween().set_parallel(true)
            success_tween.tween_property(button, "scale", Vector2(1.4, 1.4), 0.2)
            success_tween.tween_property(button, "modulate", Color(0, 1, 0), 0.2)
            success_tween.chain().tween_property(button, "scale", Vector2(1.0, 1.0), 0.2)
            success_tween.tween_property(button, "modulate", Color(1, 1, 1), 0.2)
            await success_tween.finished
            start_round()
      else:
            # Apply effects only to the pressed button
            var error_tween = create_tween().set_parallel(true)
            error_tween.tween_property(button, "scale", Vector2(0.9, 0.9), 0.15)
            error_tween.tween_property(button, "modulate", Color(1, 0.5, 0.5), 0.15)
            error_tween.chain().tween_property(button, "scale", Vector2(1.0, 1.0), 0.15)
            error_tween.tween_property(button, "modulate", Color(1, 1, 1), 0.15)
            await error_tween.finished
            is_transitioning = false
            start_idle_pulse()
            # Use the same reliable audio playback for errors
            play_audio(current_good_item)
func show_victory():
      $SFX.volume_db = Global.voice_volume
      var finishsound = preload("res://audio/effects/winning-218995.mp3")
      $SFX.stream=finishsound
      $SFX.play()
      var champion_scene = preload("res://scenes/victory_screen_wam.tscn")
      var champion_instance = champion_scene.instantiate()
      add_child(champion_instance)
func _on_main_menu_pressed() -> void:
      Global.good_obstacles_collected = 0
      Global.showad = false
      Global.victory = false
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
```

createnewtopic.gd:

```
extends Control
@onready var topic_name_input = $MarginContainer/VBoxContainer/InputTopicName
@onready var item_name_input = $MarginContainer/VBoxContainer/HBoxContainer/InputItemName
@onready var select_image_button = $MarginContainer/VBoxContainer/HBoxContainer/AddImage
@onready var image_path_label = $MarginContainer/VBoxContainer/HBoxContainer/ImagePath
@onready var select_audio_button = $MarginContainer/VBoxContainer/HBoxContainer/AddAudio
@onready var audio_path_label = $MarginContainer/VBoxContainer/HBoxContainer/AudioPath
@onready var add_item_button = $MarginContainer/VBoxContainer/HBoxContainer/AddItem
@onready var scroll_container = $MarginContainer/VBoxContainer/ScrollContainer  #
Reference to the ScrollContainer
@onready var list_vbox = $MarginContainer/VBoxContainer/ScrollContainer/VBoxContainer
@onready var item_display_container =
$MarginContainer/VBoxContainer/ScrollContainer/VBoxContainer/ItemDisplayContainer
@onready var save_button = $MarginContainer/VBoxContainer/SaveTopic
@onready var start_new_topic_button = $MarginContainer/VBoxContainer/StartNewTopic  # New
button to start a new topic
@onready var topic_manager = $MarginContainer/VBoxContainer/ManageTopics
@onready var file_dialog = $FileDialog
@onready var restriction_message = $MarginContainer/VBoxContainer/RestrictionMessage
@onready var credits_popup = $CreditsPopup
```

<<OwlyFly V1.0>>

```
@onready var printlabel = $printlabel
@onready var debug_timer = $DebugTimer
var languages = {
	0: {  # English
		"Tips": "Tips",
		"MainMenu": "Main Menu",
		"StartTopic": "Start New Topic",
		"EnterTopic": "Enter Topic Name",
		"EnterItem": "Enter Item Name",
		"AddImage": "Add Image",
		"AddAudio": "Add Audio",
		"AddImageLabel": "Path Image",
		"AddAudioLabel": "Path Audio",
		"AddItem": "Add Item",
		"RestrictionMessage1": "Demo restriction: Only one topic with three items is
allowed.",
		"RestrictionMessage2": "Corrupt topic file: ",
		"RestrictionMessage3": "Please, delete it.",
		"RestrictionMessage4": "Maximum of three items allowed.",
		"RestrictionMessage5": "A topic must have at least two items.",
		"RestrictionMessage6": "Topic name cannot be empty.",
		"SaveTopic": "Save Topic",
		"Close": "Close",
		"HelpText": "Make sure that you allow Owly Fly! to access the storage in your
phone:\n
Settings / Apps (/manage apps) / Owly Fly! / Permissions (app permissions)\n
Square images will have better results, since the game automatically will stretch it\n
To generate mp3 with the sounds, you can use websites like:\n
https://voicegenerator.io/\n
Also, the audios should not be longer than 3 seconds (enough for short sentences)."
	},
	1: {  # Chinese
		"Tips": "提示",
		"MainMenu": "主菜单",
		"StartTopic": "开始新主题",
		"EnterTopic": "输入主题名称",
		"EnterItem": "输入项目名称",
		"AddImage": "添加图片",
		"AddAudio": "添加音频",
		"AddImageLabel": "图片路径",
		"AddAudioLabel": "音频路径",
		"AddItem": "添加项目",
		"RestrictionMessage1": "演示限制：只允许一个包含三个项目的主题。",
		"RestrictionMessage2": "损坏的主题文件：",
		"RestrictionMessage3": "请删除它。",
		"RestrictionMessage4": "最多允许三个项目。",
		"RestrictionMessage5": "一个主题必须至少有两个项目。",
		"RestrictionMessage6": "主题名称不能为空。",
		"SaveTopic": "保存主题",
		"Close": "关闭",
		"HelpText": "请确保允许 Owly Fly! 访问手机存储：\n
		设置 / 应用 (/管理应用) / Owly Fly! / 权限 (应用权限)\n\n
		正方形图片效果更好，因为游戏会自动拉伸\n\n 要生成包含声音的 mp3，您可以使用以下网站：\n\n
```

```
            https://voicegenerator.io/\n\n 此外，音频不应超过 3 秒（足以用于短句）。"
      }
}
var current_language # Default to English
var items = []
var current_item = {}  # Correct type
var current_topics = []
var pressed
var current_file_type: String = ""
func _ready():
      pressed = 0
      current_language = Global.language
      $AudioSFX.volume_db = Global.voice_volume
      add_item_button.disabled = true  # Initially disable the add item button
      restriction_message.visible = false  # Initially hide the restriction message
      $MarginContainer/VBoxContainer/MainMenu.text =
languages[current_language]["MainMenu"]
      $MarginContainer/VBoxContainer/StartNewTopic.text =
languages[current_language]["StartTopic"]
      $MarginContainer/VBoxContainer/TopicName.text =
languages[current_language]["EnterTopic"]
      $MarginContainer/VBoxContainer/HBoxContainer/ItemName.text =
languages[current_language]["EnterItem"]
      $MarginContainer/VBoxContainer/HBoxContainer/AddImage.text =
languages[current_language]["AddImage"]
      $MarginContainer/VBoxContainer/HBoxContainer/ImagePath.text =
languages[current_language]["AddImageLabel"]
      $MarginContainer/VBoxContainer/HBoxContainer/AddAudio.text =
languages[current_language]["AddAudio"]
      $MarginContainer/VBoxContainer/HBoxContainer/AudioPath.text =
languages[current_language]["AddAudioLabel"]
      $MarginContainer/VBoxContainer/HBoxContainer/AddItem.text =
languages[current_language]["AddItem"]
      $MarginContainer/VBoxContainer/SaveTopic.text =
languages[current_language]["SaveTopic"]
      $Tips.text = languages[current_language]["Tips"]
      $CreditsPopup/VBoxContainer/CloseButton.text = languages[current_language]["Close"]
      _disable_all_buttons()
      _check_create_new_topic_button()
      _configure_item_list_scroll()
      if OS.get_name() == "Android" and Engine.has_singleton("GodotSAF"):
            var saf = Engine.get_singleton("GodotSAF")
            saf.connect("file_selected", self._on_file_selected_safe)
            saf.connect("copy_result", self._on_copy_result)
            debug_timer.start(1.0)
func _check_create_new_topic_button():
      current_topics = topic_manager.get_custom_topic_names()
      if not Global.is_premium_user and current_topics.size() > 0:
            _show_restriction_message(languages[current_language]["RestrictionMessage1"])
      else:
            _hide_restriction_message()
func _show_restriction_message(message: String):
      restriction_message.text = message
      restriction_message.visible = true
      _disable_all_buttons()
func _hide_restriction_message():
```

<<OwlyFly V1.0>>

```
	restriction_message.text = ""
	restriction_message.visible = false
func _disable_all_buttons():
	topic_name_input.editable = false
	item_name_input.editable = false
	select_image_button.disabled = true
	select_audio_button.disabled = true
	add_item_button.disabled = true
	save_button.disabled = true
func _enable_all_buttons():
	topic_name_input.editable = true
	item_name_input.editable = true
	select_image_button.disabled = false
	select_audio_button.disabled = false
	add_item_button.disabled = false
	save_button.disabled = false
	_check_enable_add_item_button()
func _configure_item_list_scroll():
	list_vbox.custom_minimum_size = Vector2(0, 200)
	item_display_container.custom_minimum_size = Vector2(0, 200)
	list_vbox.size_flags_vertical = Control.SIZE_EXPAND_FILL
func _reset_current_item_fields():
	current_item = {}
	item_name_input.clear()
	image_path_label.text = ""
	audio_path_label.text = ""
	add_item_button.disabled = true
func _reset_all_fields():
	topic_name_input.clear()
	items.clear()
	_reset_current_item_fields()
	clear_item_display_container()
func clear_item_display_container():
	for child in item_display_container.get_children():
		item_display_container.remove_child(child)
		child.queue_free()
func _on_manage_topics_item_selected(index: int) -> void:
	current_topics = topic_manager.get_custom_topic_names()  # Ensure current_topics is
updated
	var topic_name = current_topics[index]
	topic_name_input.text = topic_name  # Set the topic name in the input field
	var topic_data = load_existing_topic(topic_name)
	if "status" in topic_data and topic_data["status"] == "corrupt":
		_show_restriction_message(languages[current_language]["RestrictionMessage2"]
+ topic_name + languages[current_language]["RestrictionMessage3"])
	else:
		restriction_message.visible = false
		items = topic_data["items"]
		update_items_display()
		if Global.is_premium_user:
			_enable_all_buttons()
func _on_manage_topics_topic_deleted() -> void:
	current_topics = topic_manager.get_custom_topic_names()  # Ensure current_topics is
updated
	if not Global.is_premium_user and current_topics.size() == 0:
		restriction_message.text = ""
```

<<OwlyFly V1.0>>

```
	_reset_all_fields()
	update_items_display()  # Refresh the items display
	_disable_all_buttons()
	if not Global.is_premium_user and current_topics.size() == 0:
		_enable_all_buttons()
func _on_add_image_pressed() -> void:
	current_file_type = "image"
	if OS.get_name() == "Android":
		if Engine.has_singleton("GodotSAF"):
	Engine.get_singleton("GodotSAF").openFilePicker("image/*")
	else:
		# Windows code remains untouched
		file_dialog.filters = ["*.png", "*.jpg", "*.jpeg"]
		file_dialog.popup_centered()
func _on_add_audio_pressed() -> void:
	current_file_type = "audio"
	if OS.get_name() == "Android":
		if Engine.has_singleton("GodotSAF"):
	Engine.get_singleton("GodotSAF").openFilePicker("audio/*")
	else:
		# Windows code remains untouched
		file_dialog.filters = ["*.mp3"]
		file_dialog.popup_centered()
func _on_file_selected_safe(file_uri: String):
	if current_file_type.is_empty():
		debug_log("Error: File type not set!")
		return
	var dest_filename = "%s_%d.%s" % [current_file_type,
Time.get_unix_time_from_system(), file_uri.get_file().get_extension()]
	Engine.get_singleton("GodotSAF").copyFileToAppDir(file_uri, dest_filename)
func _on_copy_result(success: bool, message: String):
	if success:
		var dest_path = "user://%s" % message
		match current_file_type:
			"image":
				current_item["image"] = dest_path
				image_path_label.text = dest_path.get_file()
			"audio":
				current_item["audio"] = dest_path
				audio_path_label.text = dest_path.get_file()
		_try_enable_add_button()
	else:
		debug_log("Copy failed: " + message)
func _try_enable_add_button():
	var has_image = current_item.has("image")
	var has_audio = current_item.has("audio")
	var has_name = !item_name_input.text.is_empty()
	add_item_button.disabled = !(has_image && has_audio && has_name)
	debug_log("Add button check - Image: %s, Audio: %s, Name: %s" % [has_image,
has_audio, has_name])
func _on_file_dialog_file_selected(path: String) -> void:
	var extension = path.get_extension().to_lower()
	if extension == "png" or extension == "jpg" or extension == "jpeg":
		current_item["image"] = path
		image_path_label.text = path.get_file()
	elif extension == "mp3":
```

```
            current_item["audio"] = path
            audio_path_label.text = path.get_file()
      _check_enable_add_item_button()
func _check_enable_add_item_button():
      if current_item.has("image") and current_item.has("audio") and
item_name_input.text != "":
            add_item_button.disabled = false
      else:
            add_item_button.disabled = true
func _on_add_item_pressed() -> void:
      $AudioSFX.play()
      if not Global.is_premium_user and items.size() >= 3:
            add_item_button.disabled = true
            restriction_message.visible = true
            restriction_message.text = languages[current_language]["RestrictionMessage4"]
            return
      current_item["name"] = item_name_input.text
      items.append(current_item)
      update_items_display()
      _reset_current_item_fields()
      if items.size() == 1:
            topic_name_input.editable = false
func update_items_display():
      clear_item_display_container()
      for item in items:
            var hbox = HBoxContainer.new()
            hbox.add_child(_create_label("Name: " + item["name"]))
            hbox.add_child(_create_label("Image: " + item["image"].get_file()))
            hbox.add_child(_create_label("Audio: " + item["audio"].get_file()))
            item_display_container.add_child(hbox)
func _create_label(text: String) -> Label:
      var label = Label.new()
      label.text = text
      return label
func _on_save_topic_pressed() -> void:
      $AudioSFX.play()
      if items.size() < 2:
            _show_restriction_message(languages[current_language]["RestrictionMessage5"])
            return
      var topic_name = topic_name_input.text
      if topic_name == "":
            _show_restriction_message(languages[current_language]["RestrictionMessage6"])
            return
      save_topic_to_file(topic_name, items)
      Global.topics_created += 1
      _check_create_new_topic_button()  # Re-check the creation condition after saving
      _reset_all_fields()
      topic_manager._populate_topic_list()  # Refresh the topic list after saving
      _hide_restriction_message()
func save_topic_to_file(topic_name, topic_items):
      var topic_data = {}
      if FileAccess.file_exists("user://custom_topics/" + topic_name + ".tres"):
            topic_data = load_existing_topic(topic_name)
      else:
            topic_data = {"topic_name": topic_name, "items": []}
      var existing_items = topic_data["items"]
```

<<OwlyFly V1.0>>

```
	for item in topic_items:
		var found = false
		for existing_item in existing_items:
			if existing_item["name"] == item["name"] and existing_item["image"] ==
item["image"] and existing_item["audio"] == item["audio"]:
				found = true
				break
		if not found:
			existing_items.append(item)

			print("Added item:", item)

	var dir = DirAccess.open("user://custom_topics")
	if dir == null:
		dir = DirAccess.open("user://")
		if dir != null:
			dir.make_dir_recursive("user://custom_topics")
	for item in topic_items:
		var image_src = item["image"]
		var image_dest = "user://custom_topics/" + image_src.get_file().get_file()
		copy_file(image_src, image_dest)
		item["image"] = image_src.get_file().get_file()
		var audio_src = item["audio"]
		var audio_dest = "user://custom_topics/" + audio_src.get_file().get_file()
		copy_file(audio_src, audio_dest)
		item["audio"] = audio_src.get_file().get_file()
	topic_data["items"] = existing_items
	var file = FileAccess.open("user://custom_topics/" + topic_name + ".tres",
FileAccess.WRITE)
	file.store_var(topic_data)
	file.close()
	print("Topic data after saving:", topic_data)
	print("Saved topic with", topic_data["items"].size(), "items.")
func copy_file(src, dest):
	var src_file = FileAccess.open(src, FileAccess.READ)
	if not src_file:
		print("Failed to open source file: " + src)
		return
	var dest_file = FileAccess.open(dest, FileAccess.WRITE)
	if not dest_file:
		print("Failed to open destination file: " + dest)
		src_file.close()
		return
	dest_file.store_buffer(src_file.get_buffer(src_file.get_length()))
	src_file.close()
	dest_file.close()
func load_existing_topic(topic_name: String) -> Dictionary:
	var file = FileAccess.open("user://custom_topics/" + topic_name + ".tres",
FileAccess.READ)
	if not file:
		print("Failed to load existing topic file: " + topic_name)
		return {"status": "corrupt"}
	var topic_data = file.get_var()
	file.close()
	if topic_data == null:
		print("Corrupt topic file: " + topic_name)
```

<<OwlyFly V1.0>>

```
            return {"status": "corrupt"}
      print("Loaded topic data:", topic_data)
      print("Loaded topic with", topic_data["items"].size(), "items.")
      return topic_data
func _on_main_menu_pressed() -> void:
      if pressed==0:
            pressed=1
            $AudioSFX.play()
            await $AudioSFX.finished
      get_tree().change_scene_to_file("res://scenes/mainmenu.tscn")
func _on_start_new_topic_pressed() -> void:
      $AudioSFX.play()
      _reset_all_fields()
      if not Global.is_premium_user and current_topics.size() > 0:
            _show_restriction_message(languages[current_language]["RestrictionMessage1"])
      else:
            _enable_all_buttons()
func _on_tips_pressed() -> void:
      $AudioSFX.play()
      $CreditsPopup.visible=true
      $CreditsPopup/VBoxContainer/CreditsText.text=languages[current_language]["HelpText"
]
      credits_popup.popup_centered()
      $CreditsPopup.popup_centered()
      $CreditsPopup.popup_centered()
      $CreditsPopup.popup_centered()
      $CreditsPopup.visible=true
func _on_close_button_pressed() -> void:
      $CreditsPopup.visible=false
func debug_log(message: String) -> void:
      printlabel.text += message + "\n"  # Append the message to the label
func _on_check_name_timer_timeout() -> void:
      _try_enable_add_button()
```